

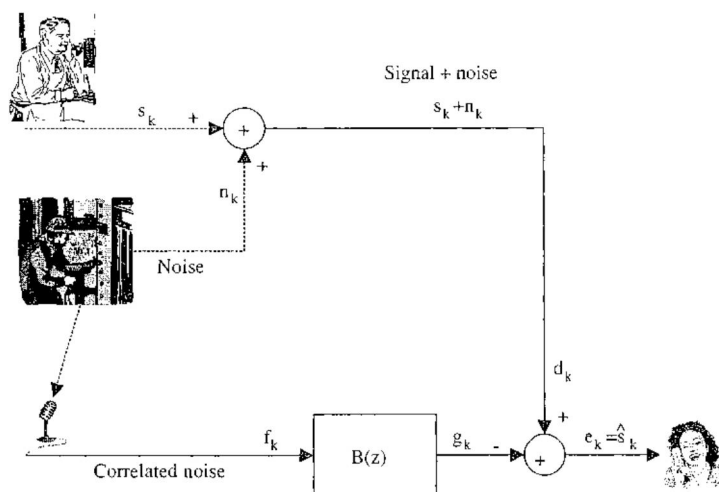


FIGURE 8.16

Least-squares interference canceling. Loud shop noise is added to the speaker's voice. Using a correlated version of the noise without the speech, the interference-canceling filter, $B(z)$, produces an output that cancels most of the noise. The filter weights are chosen to minimize the total squared value of the output, e_k .

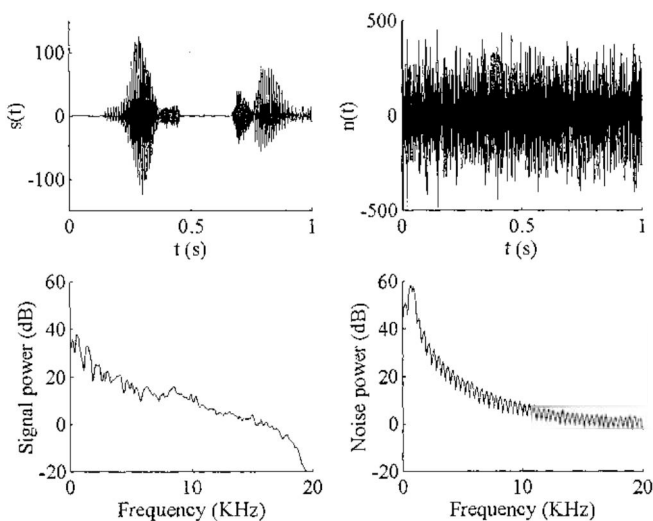
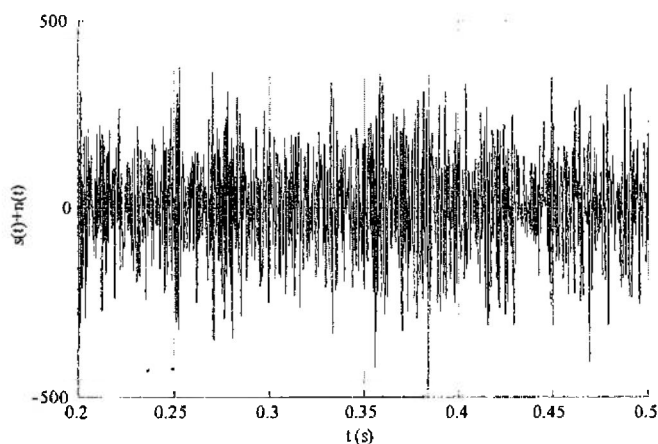
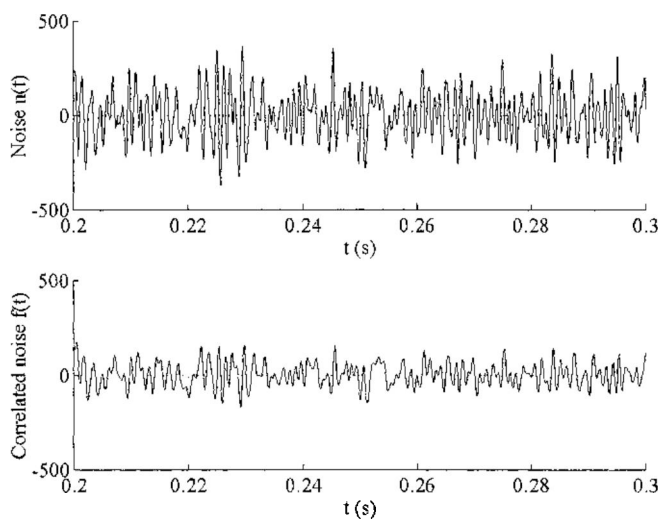


FIGURE 8.17

Speech signal, $s(t)$, and noise, $n(t)$, with power density spectra, for the interference-canceling example. The signal in this case is around 20 dB below the noise level.

**FIGURE 8.18**

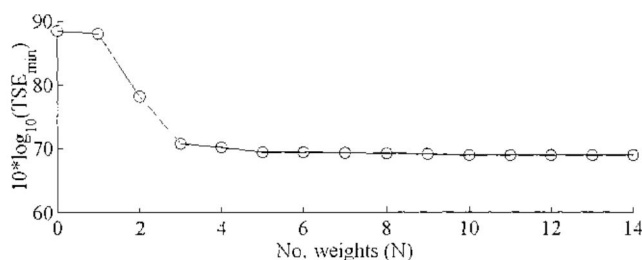
A segment of signal plus noise, illustrating a poor signal-to-noise ratio.

**FIGURE 8.19**

Segments of noise and correlated noise.

The combined signal is shown in Figure 8.18, in which we can observe that the low SNR in this case tends to obscure the speech component almost completely to the eye. The human ear is more adept than the eye at separating the two components, but even a good listener would lose most of the speech when this much noise is added.

Short (0.1-second) segments of the noise, $n(t)$, and correlated noise, $f(t)$, are shown in Figure 8.19. In this idealized example, the correlated noise is

**FIGURE 8.20**

Minimum TSE versus number of weights (N) in the interference-canceling example, indicating that the SNR increases only slightly as N increases beyond 4 or 5.

a linear transformation of the noise. In reality, we must usually accept at least a small degree of independence between the measured noise and the actual noise added to the signal.

As in previous examples, the optimal weight vector computation is described in Table 8.2. The required MATLAB expressions, in which N is the number of weights, are

$$\begin{aligned} \mathbf{Rff} &= \text{autocovar_mat}(\mathbf{f}, N); \\ \mathbf{Rfd} &= \text{crosscovar}(\mathbf{f}, \mathbf{s} + \mathbf{n}, N); \\ \mathbf{b} &= \mathbf{Rff} \backslash \mathbf{Rfd}; \end{aligned} \quad (8.41)$$

The TSE, computed as in Table 8.2 for increasing values of N , is plotted in dB in Figure 8.20. The TSE is seen to decrease slowly after $N = 4$ or 5. In practice, because quantities such as speech quality or image quality require subjective judgments, the choice of N may best be made on the basis of "best listening" or "best looking," rather than on the basis of the TSE.

In any case, we can see that least-squares interference canceling is potentially capable of producing a large increase in the SNR. The improvement is illustrated in the final figure, Figure 8.21, which shows first the noisy signal, $s(t) + n(t)$, then the system output, $e(t) = \hat{s}(t)$, for $N = 5$ and $N = 12$, and finally, the original speech signal, $s(t)$. Thanks to DSP, the listener in Figure 8.16 is able to get the message.

8.10 Linear Prediction and Recovery

In Chapter 4 (Figure 4.8), we discussed how *inverse filtering* could be used to recover a filtered signal. In Figure 8.22, we illustrate an application of inverse filtering together with linear prediction in Figure 8.1. In this application, when $B(z)$ is chosen to minimize the total squared error, the predictor removes predictable (correlated) components from the input d , leaving a

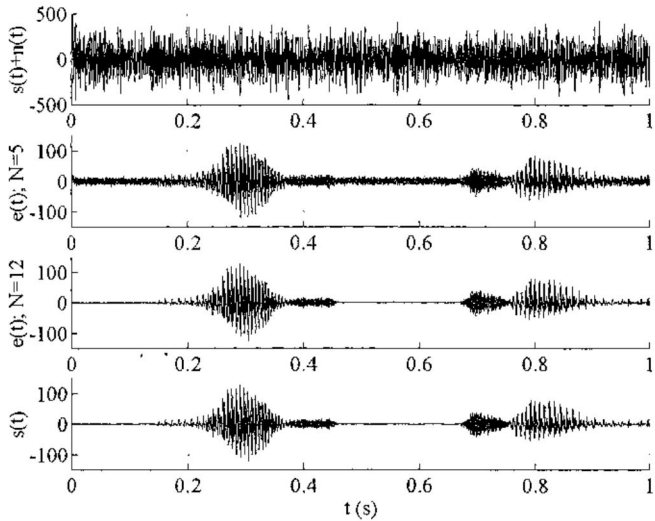


FIGURE 8.21
Final plot for the interference-canceling example, showing (1) the received signal without canceling, (2,3) the received signal with canceling using $N = 5$ and 12 weights, and (4) the original speech signal without noise.

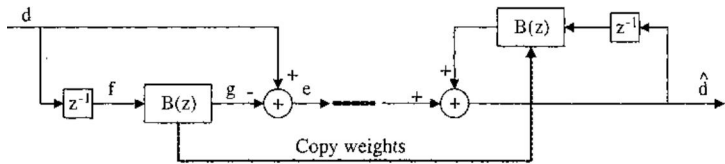
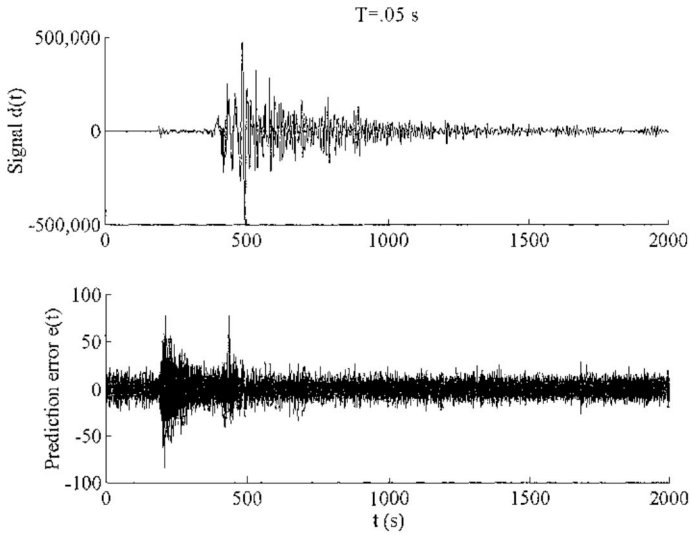


FIGURE 8.22
Waveform compression and recovery using least-squares prediction and inverse prediction. In this application, the weight vector, b , and the prediction error, e , contain all the information needed to reconstruct the original signal, d , except possibly for roundoff errors.

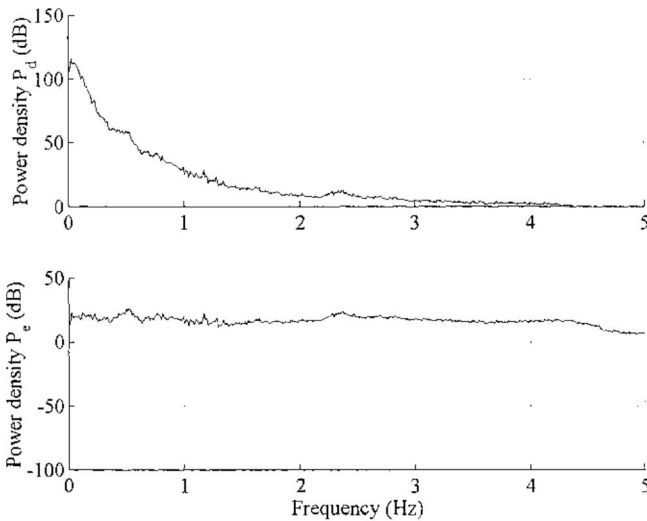
smaller residue, e . The residue becomes the compressed or *encoded* version of d , and the operation is called *linear predictive coding*.

The recovery of the original signal, d , from e is shown on the right in Figure 8.22. The recovery follows the description of inverse filtering in Chapter 4, and (4.32) in particular. Recovery requires only knowledge of the weight vector, b , and the prediction error vector, e . With the exception of roundoff errors, $\hat{d}$ and d should be identical.

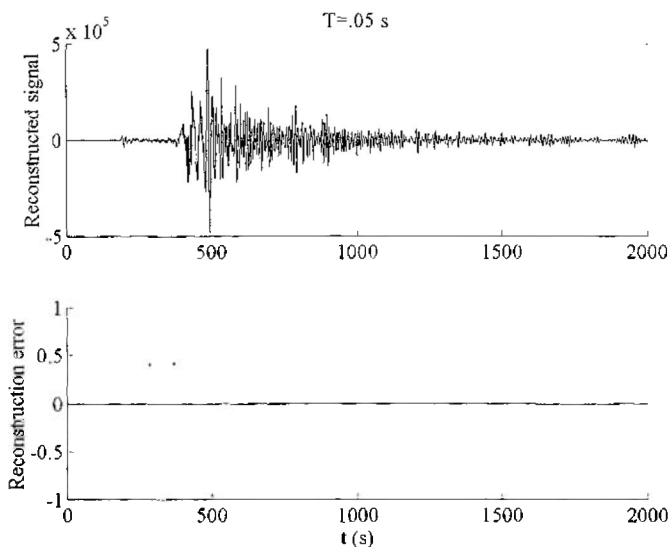
An example of predictive compression and recovery is given in Figures 8.23 through 8.25. The upper part of Figure 8.23 is a plot of 40,000 samples of a seismic event recorded by the USGS Seismological Laboratory at Albuquerque, NM. The sampling rate was 20 samples/s, so the plot spans 2000 s. The lower part of Figure 8.23 is a plot of the prediction error using a predictor

**FIGURE 8.23**

Short-period seismic event recorded by the USGS Albuquerque Seismological Laboratory, NM, and prediction error using a least-squares predictor with $N = 6$ weights. The entire prediction error, e , is shown with the exception of $e(1 : N)$. Note the different vertical scales on the two plots.

**FIGURE 8.24**

Power density spectra of the seismic waveform (upper plot) and the prediction error without startup values (lower plot). The vertical range is the same in both plots to emphasize the whitening effect of least-squares linear prediction.

**FIGURE 8.25**

Reconstructed signal, $\hat{d}$, and reconstruction error, $d - \hat{d}$. In this example, even though computations are made with a floating-point processor, the accuracy is sufficient to produce a perfect reconstruction.

with $N = 6$ weights. Notice the difference in vertical scales on the two plots. The prediction error is seen to be much smaller and gives the appearance of being more random than the upper plot.

It is remarkable, but not unusual, that a such a small linear predictor is capable of subtracting so much content in this way from a recorded signal. The reason for prediction working so well on many instrumentation and other waveforms is not hard to find. In Section 8.4, we saw that two weights were required to cancel a sinusoidal component at a given frequency with arbitrary amplitude and phase. Thus, it is reasonable that, when the input signal is nonwhite and is concentrated in one or a few narrow frequency bands, a small linear predictor can remove a large part of the signal and leave a small, white residue.

This *whitening* effect is seen in the power spectra of d and e in Figure 8.24. The spectrum of d is seen to be concentrated at low frequencies and to have a large total power. The spectrum of e , on the other hand, has a relatively small total power and, when plotted over the same range of 150 dB, looks almost flat (white).

As indicated in Figure 8.22, recovery requires knowledge of the weight vector, b , and the prediction error, e . The recovery process is an application of inverse filtering described in Chapter 4, (4.32). The reconstructed signal, $\hat{d}$, and the reconstruction error, $d - \hat{d}$, are plotted in Figure 8.25. In this case, the reconstruction is exact, and the error is zero.

The recovery operation and the predictive coding operation are both accomplished easily using MATLAB functions we have already described, as follows, in this case with $K = 40,000$ and $N = 6$:

Predictive Coding	Recovery
$f=[0; d(1:K-1)];$	$dhat=filter(1,[1 -b'],e)$
$Rff=autocovar_mat(f,N);$	
$rfd=crosscovar(f,d,N);$	
$b=Rff \backslash rfd;$	
$e=d-filter(b,1,f);$	

Without going into too much detail, we can easily see the application to signal compression in this example. If integer samples of the original waveform, d , are stored or transmitted without the use of any other coding scheme, we can see in Figure 8.23 that a range of 10^6 is required, that is, 20 bits/sample. Because d consists of 40,000 samples, 800,000 bits are required. If the prediction data is used instead, then we need to store or transmit the weight vector, b , and the prediction error, e . The first N values of e (where N is the number of weights) require 20 bits each, but afterwards, as seen in Figure 8.23, a range of only 200, or 8 bits/sample, is required. Thus, the total number of compressed data bits with $N = 6$ in this example is

$$\text{Compressed bits} = 6(20) + 6(20) + 39994(8) = 320192 \quad (8.42)$$

The *compression ratio* in this case is about 2.5, meaning that 2.5 times as many bits are required to describe the signal in its original form, compared with the compressed version.

Predictive coding has been applied in this manner to many kinds of signals and data, including speech and image data, telemetry, etc., and applied in many areas besides compression. Encryption was mentioned in Chapter 4, for example, where the weight vector becomes the *encryption key*, and the prediction error is sent in the clear.

Another application, used in spectral estimation, is called *prewhitening*. If a power spectrum is estimated using the methods described in Chapter 7, and if the signal contains one or more large narrowband components, these may bias the power density estimates at other frequencies. If so, the prewhitening operation may be used to reduce the bias. Suppose x is the signal vector, and y is the prediction error after passing x through a least-squares linear predictor. Then the transform relationship is

$$Y(z) = (1 - B(z))X(z) \quad (8.43)$$

Taking the squared magnitude of both sides of (8.43) and then applying (8.23), we have

$$Y'(z)Y(z) = |1 - B(z)|^2 X'(z)X(z), \quad \text{or} \quad \Phi_{yy}(z) = |1 - B(z)|^2 \Phi_{xx}(z) \quad (8.44)$$

From this, we conclude

$$\Phi_{xx}(z) = \frac{\Phi_{yy}(z)}{|1 - B(z)|^2} \quad (8.45)$$

Thus, the power spectrum of x may be estimated from the spectrum of the whitened signal, y , and the power gain of the least-squares predictor.

8.11 Effects of Independent Broadband Noise

In many practical design cases, noise is added to the signals with properties that are used to design least-squares systems, that is, to the vectors d and f in Figure 8.6. The noise could be *plant noise* added to d as shown in Figure 8.2, or independent *channel noise* associated with the unknown channel and added to f as shown in Figure 8.3, or *uncorrelated noise* added to either s or f in Figures 8.4 and 8.16, and so on. Our purpose, in this section, is to derive the general effects of noise in least-squares systems, assuming the noise is independent, has a broad spectrum, and is *additive*, that is, adds to the signals. These assumptions are somewhat restrictive, but nevertheless, cover many cases of interest, and the approach used here is applicable with modifications if the noise is not independent or does not have a flat power spectrum.

The noisy least-squares design configuration is illustrated in Figure 8.26. It is the same as Figure 8.6 with noise added to the signals f and d . Either of the two noise sequences, n or m , or both, may be present depending on the application of Figure 8.26. If the noises are independent of the signals, and

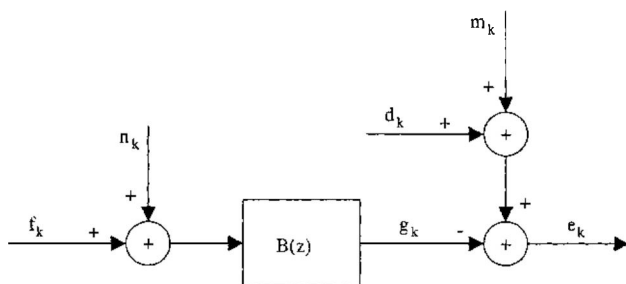


FIGURE 8.26

The essential components of the least-squares system with independent broadband noise, m_k and n_k , added to the signals d_k and f_k , respectively.

of each other, the cross-correlation and cross-spectral terms are zero. That is, for example,

$$\begin{aligned}\varphi_{f+n, f+n}(i) &= E[(f_k + n_k)(f_{k+i} + n_{k+i})] \\ &= E[f_k f_{k+i}] + E[n_k n_{k+i}] = \varphi_{ff}(i) + \varphi_{nn}(i)\end{aligned}\tag{8.46}$$

The second line in (8.46) follows, because, if f and n are independent, the expected cross-products in the first line are zero. Furthermore, when the noise is white, as we will assume here, $\varphi_{nn}(i)$ is zero except at $i = 0$; hence the corresponding autocorrelation matrix, Φ_{nn} , is diagonal.

Therefore, if we use the noisy version in Figure 8.26 in place of Figure 8.6, we replace the autocorrelation functions with noisy versions as in (8.46), leaving the cross-correlation functions unchanged, because n and m are independent. The result is the following modification of the formula (8.4) for the MSE in terms of the weight vector b :

$$\text{MSE} = \varphi_{dd}(0) + \varphi_{nn}(0) + \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} b_i b_j \varphi_{ff}(i-j) + \varphi_{nn}(0) \sum_{i=0}^{N-1} b_i^2 - 2 \sum_{i=0}^{N-1} b_i \varphi_{df}(-i)\tag{8.47}$$

Note that the third term in this expression is the same as the second term in (8.4), and the fourth term results from diagonal autocorrelation matrix, Φ_{nn} , of the white noise.

The result in (8.47), and all that follows here, is essentially the same using either correlation or covariance functions. In terms of covariance, Table 8.3 shows how the covariance equations in Table 8.2 are modified to include additive white noise.

From the formulas in Table 8.3, we reach the following conclusions:

- Adding independent white noise to the input signal (f) is equivalent to adding a constant to the diagonal of R_{ff} , the autocovariance matrix.
- Adding independent noise to the desired signal (d) simply adds a constant to the total mean-squared error and to its minimum value, $\text{TSE}_{\min}$.

TABLE 8.3
Least-Squares Covariance Equations under Noisy Conditions (Figure 8.26)

Total-squared error (TSE)	$\text{TSE} = r_{dd}(0,0) + r_{nn}(0,0) + b'(R_{ff} + r_{nn}(0,0))b - 2b'r_{fd}$ $\text{TSE} = d' * d + m' * m + b' * (R_{ff} + n' * n * I) * b - 2 * b' * r_{fd}$
Optimal weights	$b = (R_{ff} + r_{nn}(0,0))^{-1} r_{fd}$ $b = (R_{ff} + n' * n * I) \setminus r_{fd}$
Minimum TSE	$\text{TSE}_{\min} = r_{dd}(0,0) + r_{nn}(0,0) - r'_{fd}b$ $\text{TSE}_{\min} = d' * d + m' * m - r_{fd}' * b$

If R_{ff} , the autocovariance matrix, is a diagonal matrix, then cross terms of the form $b_i b_j$ with $i \neq j$ do not appear in the MSE and TSE formulas, as seen, for example, in (8.47) with $\phi_{ff}(n-m) = 0$ for $n \neq m$, and hence, the error contours (in Figure 8.9, for example) are circular. Furthermore, the $\phi_{nn}(0)$ term in (8.47) increases the b^2 terms in the MSE formula, thereby causing the sides of the bowl-shaped error surface to become steeper. Therefore, we also conclude the following:

- Adding independent white noise to the input signal tends to steepen the sides of the performance function and to make its contours more circular.

In Table 8.3, we also note that the optimal coefficients with input noise are modified by having the increased diagonal elements in the autocovariance matrix. We note that if the input is dominated by the noise sequence n , then the optimal coefficients are just scaled values of the cross-covariance terms; that is,

$$b = r_{fd} / \phi_{nn}(0) \quad (8.48)$$

Furthermore, in the limiting case where the input is entirely independent noise, the cross-correlation terms are zero, and therefore, the weight vector, b , is zero. We also observe in Table 8.3 that the solution for optimal coefficients is not affected by the noise m added to the desired signal, which only affects the TSE by adding a constant, $r_{mm}(0, 0)$.

Thus, in these results, we have expressed the effects of independent broadband noise on least-squares system design. The results in Table 8.3 for least-squares design using covariance functions, and similar results with correlation functions, indicate that least-squares system design is a valid and useful process, even if random noise is added to the signals involved.

8.12 Exercises

1. In a least-squares interference-canceling system, suppose the filter $B(z)$ has two weights, b_0 and b_1 , that the total power in the desired signal power is σ_d^2 , and that the correlation functions are $\phi_{ff}(n)$ and $\phi_{df}(n)$.
 - a. Express the mean-squared error as a function of b_0 and b_1 .
 - b. Express the optimal weights, b_0 and b_1 , in terms of the correlation functions.
 - c. Express the minimum mean-squared error in terms of the signal statistics.

2. In Exercise 1, suppose $E[f_k f_{k+1}] = \varphi_{ff}(1) = 0$.
 - a. What are the optimal weights?
 - b. What relationship between the signal statistics must hold in order for the minimum MSE to equal zero?
3. Consider a general sinusoid, $x_k = A \sin(\omega kT + \alpha)$, sampled at K samples per cycle.
 - a. What is the average power in x_k ?
 - b. Express the autocorrelation function of x_k , and show that it is independent of the phase, α .
 - c. Show that the autocorrelation function of x_k has a period equal to that of x_k .
4. A long sequence, x_k , with a period equal to eight samples is

$$x_k = [\dots, 1, 1, 1, 1, -1, -1, -1, -1, 1, 1, \dots]$$

Plot x_k and the autocorrelation function, $\varphi_{xx}(n)$, for $-15 \leq n \leq 15$.

5. If the samples of two sequences x_k and y_k are *complex*, the correlation function $\varphi_{xy}(n)$ is defined to be $\varphi_{xy}(n) \equiv E[x_k^* y_{k+n}]$; that is, $\varphi_{xy}(n)$ is the average product of the conjugate of x_k times y_{k+n} .
 - a. Show that $\varphi_{xy}(n)$ agrees with the previous definition when x_k is a real sequence.
 - b. Show that $\varphi_{xy}'(n) = \varphi_{yx}(-n)$.
 - c. Express $\varphi_{xx}(n)$ when x_k is a sinusoidal function, $x_k = Ae^{j(2\pi k/N + \alpha)}$. Show that $\varphi_{xx}(n)$ does not depend on the phase of x_k and has a period equal to that of x_k .
6. A sequence x has samples given by $x_k = A \cos(\omega k + \alpha) + r_k$, where r_k is a uniform white random variate with variance σ_r^2 . Express the autocorrelation function, $\varphi_{xx}(n)$.
7. Suppose x_k and y_k are sinusoidal signals, either real or complex, with frequencies ω_x and ω_y . What condition must hold in order for $\varphi_{xy}(n)$ to be zero for all values of n ?
8. The predictor in Figure 8.27 is a specific example of Figure 8.1 with delay $m = 2$.
 - a. Let $s_k = \sqrt{2} \cos(2\pi k/15 + \pi/4)$. Determine matrix Φ_{ff} and vector φ_{fs} for this example.
 - b. Express the mean-squared error as a quadratic function of b_0 and b_1 .
 - c. What are the optimal values of b_0 and b_1 ?
 - d. What is the minimum mean-squared error, $E[e_k^2]$?
9. Do Exercise 8 with $s_k = \sqrt{2} \sin(2\pi k/15) + r_k$, where r_k is a uniform white random sequence in the range $[-1, 1]$. Then replace r_k with αr_k and plot the $\text{MSE}_{\min}$ vs. α in the range $[0, 10]$.

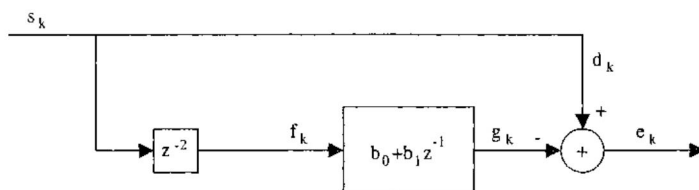


FIGURE 8.27

Two-step predictor.

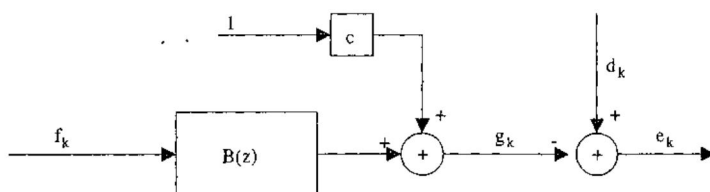


FIGURE 8.28

Least-squares system with bias weight.

10. In Exercise 8, how are matrix Φ_{ff} and vector φ_{fs} altered if we replace $(b_0 + b_1 z^{-1})$ with $(b_0 + b_1 z^{-i})$, where i is a positive integer greater than one?
11. Give a simple expression for the MSE in (8.4) when the input signal, f , is a white random sequence not necessarily independent of d .
12. A least-squares system has an adjustable "bias weight," which causes the constant c to be added to the filter output, as shown in Figure 8.28. The bias weight is used in cases where one of the signals has a nonzero mean value and the other does not. Suppose $E[f_k] = \mu_f$, $E[d_k] = 0$, $B(z)$ is a highpass filter, and the weight vector, b , is optimized. What is the optimal value of c in terms of μ_f ?
13. The noise-canceling system in Figure 8.29 is used to cancel sinusoidal interference from a broadband signal. The signal is represented as shown by a filtered version of r_k , where r_k is a uniform white random sequence with $\sigma_r^2 = 2.0$. The noise is $n_k = \sin(2\pi k/31)$, and reference signal (correlated noise) is $f_k = 2 \cos(2\pi k/31)$.
 - a. Find the power density spectrum of s_k and the total signal power, σ_s^2 .
 - b. Write an expression for the MSE, $E\{e_k^2\}$, assuming s_k and n_k are independent.
 - c. Find the optimal weights, b_0 and b_1 .
 - d. Show that the minimum MSE equals σ_s^2 .

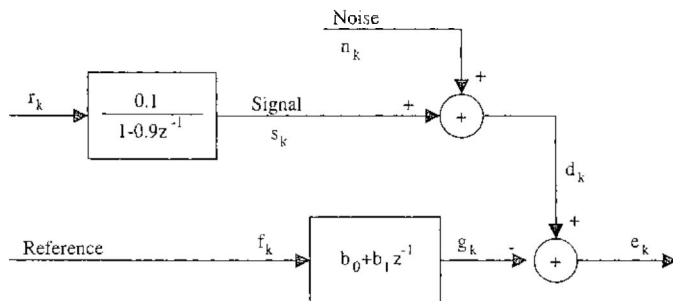


FIGURE 8.29
Noise-canceling system.

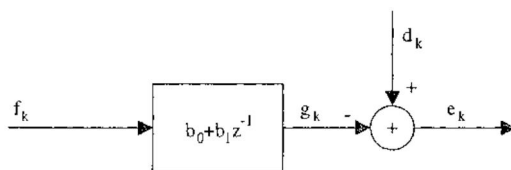


FIGURE 8.30
System with two weights.

14. For the situation described in Exercise 8, draw contours for $\text{MSE} = 1, 0.8, 0.6, 0.4,$ and 0.2 on the $b_0 - b_1$ plane. (Note: the MATLAB *contour* function is useful for this purpose.)
15. A least-squares filter is to be used to adjust the amplitude and phase of f_k to match the amplitude and phase of d_k as shown in Figure 8.30, where f_k and d_k are described as follows:

$$f_k = 5 \sin(2\pi k/32 + \pi/4); \quad d_k = \sin(2\pi k/32)$$

- a. Find b_0 and b_1 assuming the signals are unbounded.
- b. Using correlation functions with sequence lengths $K = 8, 16, 32,$ and 64 and periodic extension, print $b_0, b_1,$ and $\text{TSE}_{\min}$ for each value of K . Compare the results with part (a) and explain.
16. Do Exercise 13 using covariance functions. Find $b_0, b_1,$ and $\text{TSE}_{\min}$ for $K = 10, 100, 1000,$ and $10,000$. Compare these results with the answers to Exercise 13(c) and (d), and explain.
17. Solve the following sets of linear equations:
 - a. $b_0 + 2b_1 + 3b_2 = 8; 4b_0 + b_1 + 6b_2 = 17; 9b_0 + b_1 + b_2 = 19.$
 - b. $4b_0 + 3b_1 + 2b_2 + b_3 = 2; 3b_0 + 4b_1 + 3b_2 + 2b_3 = 0; 2b_0 + 3b_1 + 4b_2 + 3b_3 = 0; b_0 + 2b_1 + 3b_2 + 4b_3 = -2.$

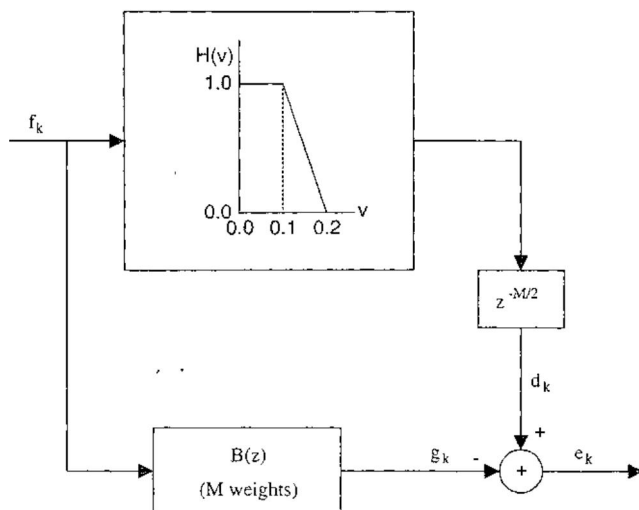


FIGURE 8.31
Least-squares FIR filter design.

- c. Five equations of the form $B = \Phi_{ff}^{-1} \Phi_{fd}$, in which the correlation functions are $\varphi_{ff}(n) = \cos(2\pi n/15)$ and $\varphi_{fd}(n) = 2 \sin(2\pi n/15)$.
 - d. $b_0 + b_1 = 1$; $b_0 + b_2 = 4$; $b_0 + b_3 = 0$; $b_0 + b_4 = 2$; $b_1 + b_2 = 3$.
18. In this exercise, we use the modeling configuration in Figure 8.13 to design a lowpass FIR filter. The concept is illustrated in Figure 8.31. The input signal, f_k , is composed of N sine waves at frequencies evenly distributed *between* zero to half the sampling rate, that is,

$$f_k = \sum_{i=1}^N \sin\left(\frac{2\pi k n}{2N+1}\right); \quad k = 0, 1, \dots, 2N$$

The desired (real) gain characteristic, $H(v)$, is applied to each of these components, and the filtered version of f_k is then delayed to produce d_k .

- a. Write an expression for d_k similar to the expression for f_k above.
- b. Write an expression for $\varphi_{ff}(n)$.
- c. With $M = 20$ and $N = 199$, find the optimal weight vector, b , the minimum TSE, and the ratio of TSE_{min} to the maximum TSE.
- d. In the upper of two subplots, plot the first halves of the DFTs of f and d vs. v in Hz-s.

- e. In the lower subplot, make a continuous plot of the filter amplitude gain, $|B(v)|$, vs. v in Hz-s.
19. Do Exercise 18 using the covariance method with signal vectors just long enough to cover one fundamental period. Comment on any differences in the results, especially in the minimum TSE.
20. Suppose the least-squares FIR filter in Figure 8.6 is a *linear-phase* filter of the following form:

$$B(z) = b_0 + \sum_{n=1}^{N-1} b_n(z^n + z^{-n})$$

- a. Express the error signal, e_k .
- b. Express the MSE, $E[e_k^2]$, in terms of correlation functions. Is the MSE a quadratic function of the weights?
- c. Derive a set of linear equations to solve for the optimal weights.
21. Design the least-squares equalizer in Figure 8.10 with $U(z)$ given by

$$U(z) = \frac{1}{1 - 1.82z^{-1} + 0.81z^{-2}}$$

For s_k , use the impulse function shown in Figure 8.11, and use $K = 50$ samples. Make a plot of the minimum TSE vs. the number of weights similar to Figure 8.12, and explain the results.

22. Design the equalizer in Exercise 21 with $N = 3$ weights and delay $m = 0$. For the input signal vector $s(1:K)$, use a white Gaussian sequence with $\sigma_s^2 = 1$ and $K = 50$. Compare your solutions for b_{opt} and TSE_{min} with the corresponding solutions in Exercise 21.
23. The *equation error* method for least-squares IIR system identification is illustrated in Figure 8.32. Note that the method produces a combined least-squares system consisting of $B(z)$, a forward model of $P(z)$, and $1 - A(z)$, an inverse model of $P(z)$. The method could be applied to model a plant with poles and zeros, to design an IIR filter using the method described in Exercise 18, to equalize a channel with zeros as well as poles, and so on.
- a. Assume f is a finite signal vector. In terms of z -transforms, show that the overall transfer function from f to e_q is $H_q(z) = P(z)(1 - A(z)) - B(z)$.
- b. Now suppose the same plant, $P(z)$, is used in Figure 8.13, and the model in that figure, $B(z)$, is replaced with a recursive model given by

$$H(z) = \frac{B(z)}{1 - A(z)}$$

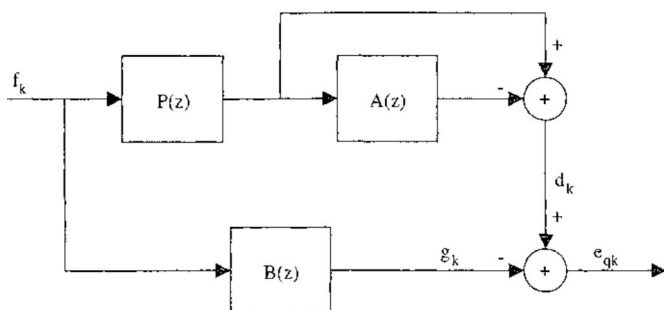


FIGURE 8.32
IIR least-squares system identification.

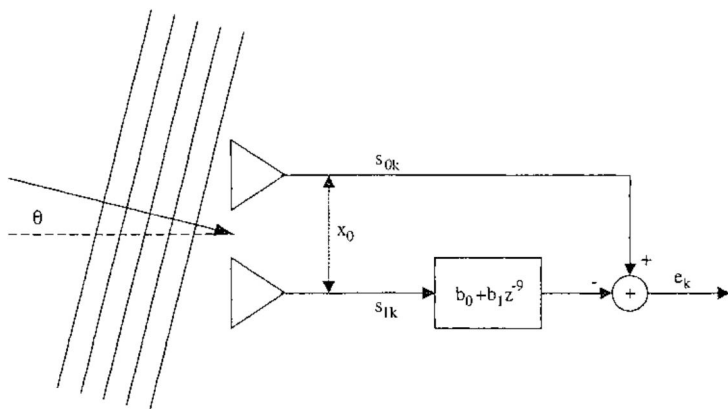


FIGURE 8.33
Direction finder.

Express $E_q(z)$ in Figure 8.32 in terms of $E(z)$ in Figure 8.13, and prove the following:

$$E_q(z) = (1 - A(z))E(z)$$

Thus, if the model in Figure 8.32 is perfect in the sense that $e_{qk} = 0$, then $e_k = 0$ in Figure 8.13 as well. Therefore, as the error approaches zero, $H(z)$ above becomes a valid least-squares IIR model of the plant in Figure 8.32.

- c. Show how to minimize the MSE, first with g fixed and then with b fixed.
- d. Let $P(z) = H(z)$ in Chapter 4, Exercise 5. Iterate, solving for $[b_0, b_1]$ with a fixed and then $[a_1, a_2]$ with b fixed, until $B(z)/(1 - A(z))$ converges to $P(z)$. Plot all five weights versus the iteration number as they converge to the weights in $P(z)$.

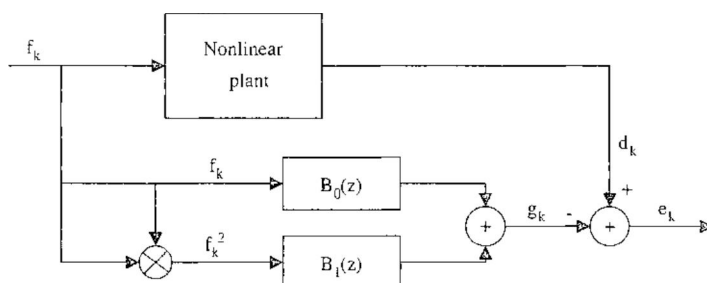


FIGURE 8.34
Least-squares nonlinear modeling.

24. A simple direction-finding system is shown in Figure 8.33. A plane wave traveling at velocity c m/s arrives with direction angle θ at two receptors spaced x_0 meters apart, such that

$$s_{0k} = A \cos\left(\frac{2\pi k}{36}\right); \quad s_{1k} = A \cos\left(\frac{2\pi(k-\Delta)}{36}\right)$$

where $\Delta = (x_0 \sin \theta)/c$ s, c being the speed of light. The second signal, s_{1k} , is sent through a *quadrature filter*, in which there is a 90-degree (nine-sample) phase delay from b_0 to b_1 .

- Express the optimal weights in terms of the signal direction, θ .
 - Describe how the system could be used as a direction finder, that is, to acquire a signal and determine the direction θ .
 - Comment on the effects of adding mutually independent white noise at the two inputs.
25. A *nonlinear* modeling system is shown in Figure 8.34. The plant being modeled is *nonlinear*, and the system attempts to construct a least-squares *polynomial model* of the plant. In this case, the degree is 2, but it is easy to see how higher degrees of f_k could be appended to the bottom of the figure and added to g_k . A bias weight could also be added as in Exercise 12 to complete the polynomial.
- Derive an expression for the TSE similar to (8.18) for Figure 8.34, and show that the error surface is a quadratic function of the weights having a single global minimum.
 - Prove that the solution for optimal weights amounts to solving a set of linear equations.
26. In Exercise 13, suppose uniform white noise with power equal to 3 is added to the reference input.
- Write the revised expression for the MSE.

- b. Find the optimal weights.
- c. Compute the minimum MSE and compare the result with σ_s^2 .

References

1. Haykin, S.S., *Adaptive Filter Theory*, 2nd ed., Prentice Hall, Englewood Cliffs, NJ, 1991.
2. Orfanidis, S., *Optimum Signal Processing: An Introduction*, Macmillan, New York, 1985.
3. Widrow, B. and Stearns, S.D., *Adaptive Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1985.
4. Blahut, R., *Fast Algorithms for Digital Signal Processing*, Addison-Wesley, Reading, MA, 1985.
5. Saywood, K., *Introduction to Data Compression*, Morgan Kaufman, San Francisco, CA, 1996.
6. Jain, A., *Fundamentals of Image Processing*, Prentice Hall, Englewood Cliffs, NJ, 1989.
7. Rabiner, L.R. and Schafer, R.W., *Digital Processing of Speech Signals*, Prentice Hall, Englewood Cliffs, NJ, 1978, chap. 8.
8. Franklin, G.F. and Powell, J.D., *Digital Control of Dynamic Systems*, Addison-Wesley, Reading, MA, 1980.
9. Marple, S.L., *Digital Spectral Analysis with Applications*, Prentice Hall, Englewood Cliffs, NJ, 1987.
10. Kay, S., *Modern Spectral Estimation: Theory and Application*, Prentice Hall, Englewood Cliffs, NJ, 1987.
11. Atal, B.S. and Hanauer, S.L., Speech analysis and synthesis by linear prediction of the speech wave, *J. Acoust. Soc. Amer.*, 50, 637, 1971.
12. Bordley, T.E., Linear predictive coding of marine seismic data, *IEEE Trans. Acoust. Speech Signal Process.*, ASSP-31, 828, Aug. 1983.
13. Clark, G.A. and Rodgers, P.W., Adaptive prediction applied to seismic event detection, *Proc. IEEE*, 69, 1166, Sep. 1981.
14. Zeidler, J.R. et al., Adaptive enhancement of multiple sinusoids in uncorrelated noise, *IEEE Trans. Acoust. Speech Signal Process.*, ASSP-26, 240, June 1978.
15. Itakura, F. and Saito, S., Digital filtering techniques for speech analysis and synthesis, *Proc. 7th International Congress on Acoustics*, 261, 1971.
16. Makhoul, J., Spectral analysis of speech by linear prediction, *IEEE Trans. Audio Electroacoust.*, AU-21, 140, June 1973.
17. Lucky, R.W., Techniques for adaptive equalization of digital communication systems, *Bell System Tech. J.*, 255, Feb. 1966.
18. Gersho, A., Adaptive equalization of highly dispersive channels for data transmission, *Bell System Tech. J.*, 55, Jan. 1969.
19. Ives, R., Near lossless compression of SAR imagery using gradient adaptive lattice filters, *Proc. 35th Asilomar Conf. on Signals, Systems, and Computers*, Nov. 2001.
20. Griffiths, L.J., Smolka, F.R., and Trembly, L.D., Adaptive deconvolution: a new technique for processing time-varying seismic data, *Geophysics*, 742, June 1977.
21. Widrow, B. et al., Adaptive noise canceling: principles and applications, *Proc. IEEE*, 63, 1692, Dec. 1975.

22. Kailath, T., Ed., Special Issue on System Identification and Time Series Analysis, *IEEE Trans. Automatic Control*, AC-19, Dec. 1974.
23. Widrow, B., Titchener, P.P., and Gooch, R.P., Adaptive design of digital filters, *Proc. ICASSP'81*, 243, Mar. 1981.
24. Wiener, N., *Extrapolation, Interpolation and Smoothing of Stationary Time Series with Engineering Applications*, Wiley, New York, 1949.
25. Levinson, N., The Wiener RMS error criterion in filter design and prediction, *J. Math. Phys.*, 25, 261, 1946.
26. Zohar, S., Toeplitz matrix inversion: the algorithm of W. F. Trench, *J. Assoc. Comput. Mach.*, 16, 592, Oct. 1969.

Adaptive Signal Processing

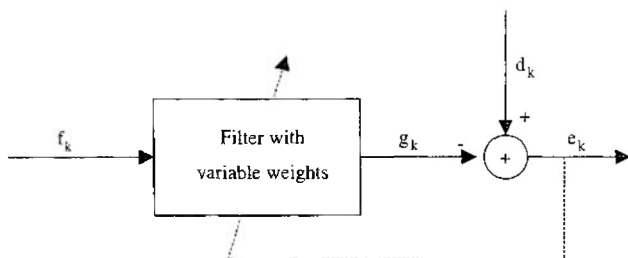
9.1 Introduction

In this chapter, which introduces a type of system we have not yet considered, we will be relying on the discussions of linear least-squares systems in Chapter 8. Adaptive signal processing systems, as introduced here, are time-varying versions of the systems in Chapter 8, in which the weights are varied to allow the systems to adapt to trends or variations in the statistical properties of nonstationary signals.

Adaptive signal processing systems appear in many different forms, including all of the different applications mentioned in Chapter 8. The diversity has been promoted by developments in microelectronics that have greatly increased the amount of computing possible in real-time DSP systems. Seismic signals at 10^2 Hz and below, speech and acoustic signals at 10^2 to 10^5 Hz, electromagnetic signals at 10^5 Hz and beyond, signals in control systems at all frequencies, as well as other similar signals appearing in time and space are all reasonable candidates for adaptive signal processing.

The *adaptive* notion derives from the human desire to emulate living systems found in nature, which adapt to their environments using various remarkable methods that we can copy only in very limited ways. In the development of adaptive systems, the journey from concept to reality has taken us away from direct emulation, somewhat like the development of manned aircraft. An adaptive signal processor usually resembles its natural counterpart about as much as, or if anything less than, an airplane resembles a bird.

Adaptive signal processing has its roots in adaptive control and in the mathematics of iterative processes, where the first attempts were made to design systems that adapt to their environments. In this chapter, we focus on the basic theory and applications of modern adaptive signal processing. Much of what we describe here is based on the work of Bernard Widrow and his colleagues,^{1-3,5,6,12,13,18,24,30,35,37,40} which began around 1960. We use Widrow's development of the geometry of adaptation,⁵ and we introduce the least-mean-square (LMS) algorithm as the simplest and most widely applicable adaptive processing scheme.

**FIGURE 9.1**

Essential elements in adaptive signal processing. These are the same as the essential elements in least-squares design, except that here the filter weights are adjusted continuously to minimize the squared error, e_k^2 .

Adaptation involves a process known as *performance feedback*,⁵ in which the performance of a system, measured in terms of the same squared error we used in Chapter 8, is used to adjust the adaptive weights. The weights may be adjusted at each step in the sampling process, or after N steps in what is known as *block adaptive processing*.

In any adaptive signal processing system, we are usually able to identify the basic structure and signals in Figure 8.5 or Figure 8.6, depending on whether the *adaptive filter* is recursive or not. Thus, we begin with the structure and signals in Figure 9.1. The variable weights are adjusted repeatedly, at regular intervals, in accordance with an *adaptive algorithm*. The adaptive algorithm usually uses, either explicitly or implicitly, the signals shown in Figure 9.1, that is, the input signal, f_k , the desired signal, d_k , and the error signal, e_k , which is the difference between d_k and g_k , the filtered version of f_k . The details of the adaptive algorithms are given in the following sections. The main point here is that the weights are adjusted continually during adaptation to reduce the mean-square error, $E[e_k^2]$, toward its minimum value. Thus, the adaptive system seeks continually to reduce the difference between the desired response, d_k , and its own response, g_k . In this way, as we shall see, the system “adapts” to its prescribed signal environment.

In this chapter, we can only hope to introduce some of the basic ideas and applications of adaptive signal processing, because the subject is so broad. Texts devoted to this subject, some of which are listed in the References, are recommended for better coverage. The text by Haykin⁴ has an especially comprehensive and complete treatment of adaptive signal processing and filter theory.

9.2 The Mean-Squared Error Performance Surface

In Chapter 8, Section 8.3, we saw that the mean squared error (MSE) was a quadratic function of the samples, $[h_k]$, of the impulse response of the least-squares filter. The function was derived in (8.4), which is repeated

here:

$$\text{MSE} = \varphi_{dd}(0) + \sum_{n=0}^{N-1} \sum_{m=0}^{N-1} b_n b_m \varphi_{ff}(n-m) - 2 \sum_{n=0}^{N-1} b_n \varphi_{df}(-n) \quad (9.1)$$

To keep the quadratic form and have a finite filter, in Chapter 8, we set the impulse response equal to b , the weight vector of an FIR filter, as we have done here in (9.1). Later in this chapter, we will consider the consequences of using an adaptive IIR filter.

In adaptive systems, (9.1) describes what is called a quadratic *performance surface*,⁵ meaning that the performance of an adaptive system may be described in terms of the geometry of this surface, which turns out to be a helpful vehicle for visualizing the adaptive process. Examples of two ways to view the performance surface with two weights were given in Chapter 8, Figures 8.8 and 8.9.

The key equations describing the performance surface were given in Table 8.2 in terms of correlation functions, that is,

$$\text{Mean-squared error: } \text{MSE} = \varphi_{dd}(0) + b' \Phi_{ff} b - 2b' \varphi_{fd} \quad (9.2)$$

$$\text{Optimal weights: } b = \Phi_{ff}^{-1} \varphi_{fd} \quad (9.3)$$

$$\text{Minimum MSE: } \text{MSE}_{\min} = \varphi_{dd}(0) - \varphi'_{fd} b \quad (9.4)$$

Under stationary conditions, the correlation functions are constant properties of the signals. If, as we assume in this chapter, the signals are *nonstationary*, then we may view the correlation functions as changing with time. In (9.2), we can see that this implies an MSE, that is, a performance surface that moves and changes shape as the correlation functions change. The goal of the adaptive signal processor is then to track these changes and continuously adjust the elements of the weight vector (b) so the MSE is kept near its minimum value.

In the sections that follow, we will consider methods for searching the performance surface and tracking the movement of the minimum MSE. The most general adaptive algorithms for searching performance surfaces under unknown signaling conditions are *random search* algorithms. One type of random search algorithm²⁴ selects a random *direction* in the performance space in which to look at each iteration and then moves in directions that happen to reduce the MSE. An even more general random search algorithm²⁵ selects random *locations* on the weight plane (or hyperplane), estimates the MSE at these points, and retains points with the lowest MSE from one iteration to the next.

In this chapter, we will focus on the two most common methods for accomplishing such adjustments in real-time adaptive systems; namely, steepest descent and sequential regression. These methods work best with

quadratic error surfaces, and therefore, are applicable to FIR adaptive filters, including the equation-error structure described in Chapter 8, Exercise 23. However, these are by no means the only methods that work, especially with other types of filters and nonquadratic error surfaces.

9.3 Searching the Performance Surface

With the types of systems and the quadratic performance surfaces we have been discussing, as well as with similar performance functions that are continuous and have continuous derivatives, we can implement powerful, deterministic (that is, nonrandom) adaptive search algorithms. In this section, we provide a basis for these algorithms in terms of the parabolic MSE in (9.2), viewing the MSE as a function of the N elements of the weight vector, b , that is, viewing the MSE as a performance surface in $(N + 1)$ -dimensional space. As discussed previously, in adaptive processing, we also view the performance surface as moving and changing shape, so the algorithms must continually adjust b to track the minimum MSE.

We will see that to track the minimum MSE in this manner, deterministic search algorithms, in one way or another, must employ an estimate of the *gradient* of the MSE performance surface. The gradient of the MSE is a vector with elements that are derivatives with respect to the weights; that is, similar to (8.5),

$$\nabla = \frac{\partial \text{MSE}}{\partial b} = \left[\frac{\partial \text{MSE}}{\partial b_0} \frac{\partial \text{MSE}}{\partial b_1} \dots \frac{\partial \text{MSE}}{\partial b_{N-1}} \right] \quad (9.5)$$

We can write the MSE gradient as the vector form of (9.5) by differentiating the MSE in (9.2) with respect to b . To differentiate the second term in (9.2), let $\delta(n)$ represent a vector of length N , with the n th element equal to one and all other elements equal to zero. Then, we have

$$\begin{aligned} \frac{\partial}{\partial b_n} \{ b' \Phi_{ff} b \} &= \delta(n) \Phi_{ff} b + b' \Phi_{ff} \delta(n) \\ &= \Phi_{ff}(n, :) b + b' \Phi_{ff}(:, n) = 2 \sum_{m=0}^{N-1} \varphi_{ff}(n-m) b_m \end{aligned} \quad (9.6)$$

The final result is due to the symmetry of Φ_{ff} , that is, $\Phi_{ff}(n-m) = \Phi_{ff}(m-n)$. Thus, the gradient vector, which is the derivative of (9.2) with respect to b , is given by

$$\nabla = 2\Phi_{ff}b - 2\varphi_{fd} \quad (9.7)$$

Setting $\nabla = 0$ produces the solution in 9.3 for optimal weights.

Now we consider a weight vector, $b(k)$, as a function of its location in the sample space. We will assume sampling takes place in time, so $b(k)$ is the weight vector and $\nabla(k)$ the gradient vector at time kT in the time domain of the adaptive processor. We denote the *optimal* weight vector in (9.3) as b_{opt} . If we multiply (9.7) by $\Phi_{ff}^{-1}/2$ and add the result to (9.3), we may obtain the following “one-step” algorithm for moving from an arbitrary point on the performance surface to the minimum MSE in a single step:

$$b_{\text{opt}} = b(k) - \frac{1}{2} \Phi_{ff}^{-1} \nabla(k) \quad (9.8)$$

This result is essentially Newton’s root-finding method⁴² applied to find the zero of a linear function (∇ in this case). It can also be derived by writing the MSE in the form of a Taylor series, as in the literature.⁴³ Given any weight vector $b(k)$ along with Φ_{ff} and the corresponding gradient $\nabla(k)$, we can move from $b(k)$ to the optimum weight vector b_{opt} in a single step by adjusting b in accordance with (9.8).

If we could apply (9.8) in practical situations such as those illustrated in Figures 8.1 to 8.4 in the previous chapter, we would always be able to adapt in one iteration to the optimal weight vector. As with natural systems, however, practical adaptive signal processing systems do not have enough information to adapt perfectly in just one iteration. There are two specific problems with (9.8) in practice, under nonstationary conditions:

1. The correlation functions change and can, at best, only be estimated.
2. The gradient must be estimated at each adaptive iteration, using local statistics.

In other words, we must work with estimated values of Φ_{ff}^{-1} and $\nabla(k)$ in (9.8).

To anticipate the use of noisy estimates of these values, we modify (9.8) to obtain an algorithm that adjusts b in smaller increments and converges to b_{opt} after many iterations. The small increments have the effect of smoothing the noise in the estimates of Φ_{ff}^{-1} and $\nabla(k)$, thus allowing the adaptive system to have stable behavior. The modified version of (9.8), which allows us to compute $b(k+1)$ in terms of $b(k)$ and a convergence parameter, u , is

$$\boxed{b(k+1) = b(k) - u \Phi_{ff}^{-1} \nabla(k)} \quad (9.9)$$

We can examine the convergence of (9.9) by first subtracting $2u$ times (9.8) from (9.9) to obtain

$$b(k+1) = (1 - 2u)b(k) + 2ub_{\text{opt}} \quad (9.10)$$

Taking the z -transform of (9.10) and solving for $B(z)$, we have

$$\begin{aligned} zB(z) &= (1 - 2u)B(z) + \frac{2uzb_{\text{opt}}}{z - 1}; \\ B(z) &= \frac{2uzb_{\text{opt}}}{(z - 1)(z - (1 - 2u))} \end{aligned} \quad (9.11)$$

Taking the inverse transform (Table 4.2, line 6), we have

$$b(k) = [1 - (1 - 2u)^k]b_{\text{opt}} \quad (9.12)$$

Here we have a formula that begins at $b(0) = 0$ and converges (conditionally) to b_{opt} at $k = \infty$. Because the formula follows a linear path on the b -plane, we may move the starting point to $b(0)$ by substituting $b(k) - b(0)$ for $b(k)$ and $b_{\text{opt}} - b(0)$ for b_{opt} . Then (9.12) becomes

$$b(k) = b_{\text{opt}} + (1 - 2u)^k(b(0) - b_{\text{opt}}) \quad (9.13)$$

Here we have a description of the relaxation of the weight vector from $b(0)$ to b_{opt} under ideal conditions, where Φ_f and $\nabla(k)$ are known at each step.

In (9.13), we can see that $b(k)$ will converge to b_{opt} only if the geometric ratio $1 - 2u$ is less than 1 in magnitude; that is,

$$\text{For convergence under ideal conditions: } 0 < u < 1 \quad (9.14)$$

In (9.13), the one-step solution is found at $k = 1$ with $u = 1/2$. With $u > 1/2$ the convergence is oscillatory, with b jumping back and forth across the bowl and converging toward b_{opt} . In practical adaptive systems with noise, values of u well below $1/2$, typically on the order of 0.01, are used, giving a convergence time constant of many iterations. We define the convergence time constant to be τ iterations, where the relaxation toward b_{opt} goes in proportion to $(1 - e^{-k/\tau})$. Then, using the linear approximation $(1 - 1/\tau)$ for $e^{-1/\tau}$ when τ is large, and comparing it with (9.13), we have

$$\begin{aligned} 1 - e^{-k/\tau} &\approx 1 - (1 - 1/\tau)^k \\ &\approx 1 - (1 - 2u)^k \end{aligned} \quad (9.15)$$

Thus, we conclude that $1/\tau = 2u$, and we have the following rule:

$$\begin{aligned} &\text{Time constant for convergence of weights under ideal conditions:} \\ &\tau_w \approx \frac{1}{2u} \text{ iterations; } \quad 0 < u < 1/2 \end{aligned} \quad (9.16)$$

In addition to weight convergence in (9.13), the convergence of the MSE to its minimum value is also often used as a performance measure in adaptive

systems. A plot of the MSE vs. the iteration number k is euphemistically called a *learning curve*.⁵ To derive a formula similar to (9.13) for the learning curve, it is convenient to use a translated weight vector, c , that allows the minimum MSE to occur at $c_{\text{opt}} = 0$:

$$c = b - b_{\text{opt}} \quad (9.17)$$

Then, noting that $b' \varphi_{fd} = \varphi'_{fd} b$, the MSE in (9.2) becomes

$$\begin{aligned} \text{MSE} &= \varphi_{dd}(0) + (c + b_{\text{opt}})' \Phi_{ff}(c + b_{\text{opt}}) - 2\varphi'_{fd}(c + b_{\text{opt}}) \\ &= \varphi_{dd}(0) + b'_{\text{opt}} \Phi_{ff} b_{\text{opt}} - 2\varphi'_{fd} b_{\text{opt}} + c' \Phi_{ff} c + c' \Phi_{ff} b_{\text{opt}} + b'_{\text{opt}} \Phi_{ff} c - 2\varphi'_{fd} c \end{aligned} \quad (9.18)$$

We can simplify this result in two steps. First, if we substitute (9.3) for the first occurrence in the second line of b'_{opt} in (9.18), the first three terms become equal to $\text{MSE}_{\min}$ in (9.4). Second, again using (9.3) in the last three terms, we can substitute $\Phi_{ff} b_{\text{opt}} = \varphi_{fd}$, and also, because Φ_{ff} is symmetric, $b'_{\text{opt}} \Phi_{ff} = \varphi'_{fd}$. Therefore, the last three terms sum to zero, and (9.18) reduces to

$$\boxed{\text{MSE} = \text{MSE}_{\min} + c' \Phi_{ff} c} \quad (9.19)$$

This is the simplest expression for the quadratic performance surface, given in terms of coordinates normalized so that the optimal weight vector, c_{opt} , is zero.

If we substitute (9.17) into (9.13), we have a simpler formula for the relaxation of the weight vector from a starting value, c_0 , to its optimum value, that is, to zero:

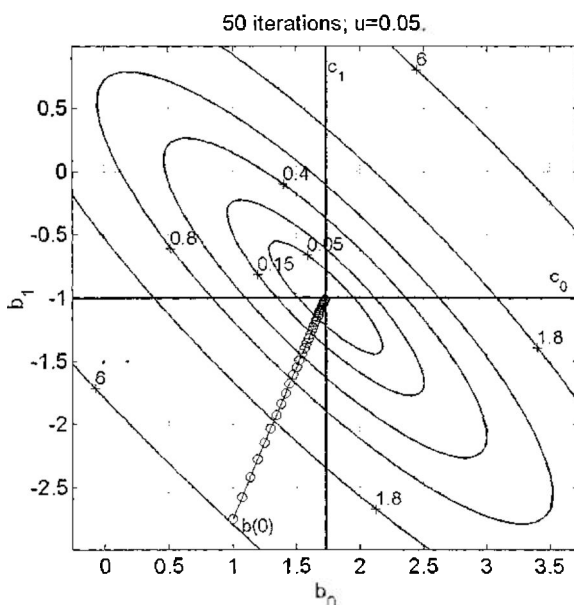
$$c(k) = (1 - 2u)^k c(0) \quad (9.20)$$

Substituting (9.20) for c in (9.19), we have a formula for the learning curve, that is, the relaxation of the MSE to its minimum, under ideal conditions:

$$\text{MSE}_k = \text{MSE}_{\min} + (1 - 2u)^{2k} c'(0) \Phi_{ff} c(0) \quad (9.21)$$

Comparing this with (9.20) or (9.13), we see that the convergence ratio is now $(1 - 2u)^2$ instead of $(1 - 2u)$. Thus, similar to (9.16), the time constant for the learning curve is

$$\boxed{\begin{aligned} &\text{Time constant for convergence of the MSE under ideal conditions:} \\ &\tau_{\text{MSE}} = \frac{\tau_w}{2} \approx \frac{1}{4u} \text{ iterations; } \quad 0 < u < 1/2 \end{aligned}} \quad (9.22)$$

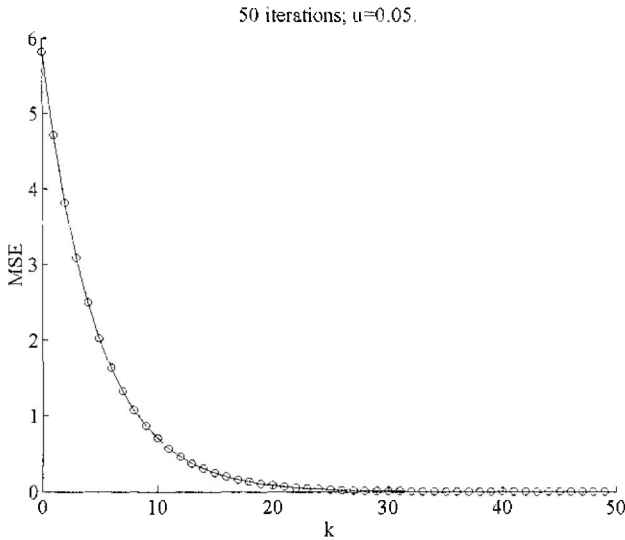
**FIGURE 9.2**

Weight convergence in the predictor described in Chapter 8, Section 8.4, with error contours illustrated in Figure 8.9. Convergence is in accordance with (9.13), with $u = 0.05$, beginning at $b(0) = [1.00, -2.75]$, and continuing to $b(49)$.

Examples of weight convergence and a learning curve are given in Figures 9.2 and 9.3. These examples illustrate ideal convergence of the weights as described by (9.9), beginning at $b(0) = [1.00; -2.75]$ and using $u = 0.05$, the latter being well below the one-step convergence value, $u = 0.5$. The examples are for the prediction example in Chapter 8, Section 8.4, with performance surface illustrated in Figures 8.8 and 8.9. In Figure 9.2, note the straight, noise-free track from $b(0)$ to $b(49)$, which is close to b_{opt} and also the translated coordinates of c with origin at b_{opt} . With $u = 0.05$, the weight convergence time constant is approximately 10 iterations in accordance with (9.16), and we note in Figure 9.2 that there is convergence after five time constants, or 50 iterations.

In Figure 9.3, we observe the corresponding exponential decrease of the MSE from its starting value of 5.8 toward the minimum value, which is zero in this case. The MSE time constant, τ_{MSE} , is approximately five iterations in accordance with (9.22), and we observe in Figure 9.3 that the approximation is valid. After four time constants, or 20 iterations in this example, the MSE has relaxed to about 1.1% of its original value, which is close to $5.8e^{-4}$.

In this section, we discussed the process of searching the quadratic performance surface. The principal formulas, (9.9) and (9.13), describe how,

**FIGURE 9.3**

Convergence of the MSE ("learning curve") corresponding with weight convergence in Figure 9.2.

under ideal conditions, the weight vector changes at each step and converges finally to its optimum value.

We must now examine how adaptive systems reach convergence under realistic conditions, with imperfect knowledge of the gradient and correlation properties of the performance surface, which may change from one iteration to the next. First, we will assume that we have little knowledge of the correlation properties and can only estimate the input signal power, that is, $\phi_{ff}(0)$. This assumption leads us to the *steepest-descent* class of algorithms and, in turn, to the widely used Widrow-Hoff *LMS algorithm*, which is the simplest and most useful algorithm in adaptive signal processing.

9.4 Steepest Descent and the LMS Algorithm

A *steepest-descent* adaptive algorithm is a formula like (9.9) for step-by-step adjustment of the weight vector, but it is unlike (9.9) in that it does not move along a straight track in the weight plane (or hyperplane) as in Figure 9.2, but rather follows the path of steepest descent down the error surface, perpendicular to the MSE contours in Figure 9.2.

One of the goals in developing such an algorithm is to revise (9.9) by removing the correlation information in Φ_{ff}^{-1} , and yet be able to control convergence as we did in (9.9) by restricting μ to the range $(0, 1/2)$. To accomplish this, we recall that Φ_{ff} is a square matrix with rows and columns that are

permutations of the autocorrelation vector, Φ_{ff} . If Φ_{ff} is nonsingular so that Φ_{ff}^{-1} exists, which is the case in all our applications, the *eigenvalues* of Φ_{ff} are developed from the following equation,⁵ in which λ_n is a scalar, I is the identity matrix, and q_n is a column *vector* (not a single element) with N elements, $q_{n0}, q_{n1}, \dots, q_{n,N-1}$:

$$[\Phi_{ff} - \lambda_n I]q_n = [0 \dots 0]' \quad (9.23)$$

This equation has nontrivial solutions for λ_n and q_n if the following determinant vanishes:

$$\det[\Phi_{ff} - \lambda_n I] = 0 \quad (9.24)$$

This is called the *characteristic equation* of Φ_{ff} . Its N solutions, designated $\lambda_0, \lambda_1, \dots, \lambda_{N-1}$, are the *eigenvalues* of Φ_{ff} . Corresponding with each distinct eigenvalue, λ_n , there is an *eigenvector*, q_n , satisfying (9.23). In (9.23), we note that q_n may be *normalized* (multiplied by a scalar) so that

$$q'_n q_n = 1; \quad n = 0, 1, \dots, N-1 \quad (9.25)$$

We will assume the eigenvalues are distinct, and the eigenvectors are normalized in this manner.

For each eigenvalue λ_n we may express the equality in (9.23) by writing

$$\Phi_{ff} q_n = \lambda_n q_n; \quad n = 0, 1, \dots, N-1 \quad (9.26)$$

From this expression, we can show that the N eigenvectors are *mutually orthogonal*. We take two versions of (9.26), one for q_m and the other for q_n :

$$\Phi_{ff} q_m = \lambda_m q_m; \quad \Phi_{ff} q_n = \lambda_n q_n$$

We multiply the first expression by q'_n and the second by q'_m :

$$q'_n \Phi_{ff} q_m = \lambda_m q'_n q_m; \quad q'_m \Phi_{ff} q_n = \lambda_n q'_m q_n$$

We transpose the first of these expressions, recalling that $\Phi'_{ff} = \Phi_{ff}$. The result is the same as the second expression; that is,

$$(q'_n \Phi_{ff} q_m)' = q'_m \Phi_{ff} q_n = \lambda_n q'_m q_n = \lambda_m q'_m q_n$$

Because the expressions are equal, and because λ_m and λ_n are different, we conclude that

$$q'_m q_n = q'_n q_m = 0; \quad n \neq m \quad (9.27)$$

and thus, the eigenvectors are mutually orthogonal.

Next, we form the *eigenvector matrix*, Q , as a square matrix with q_n as the n th column, and also Λ , a diagonal matrix with λ_n as the n th diagonal element:

$$Q = [q_0 \ q_1 \ \dots \ q_{N-1}]; \quad \Lambda = \text{diag}[\lambda_0 \ \lambda_1 \ \dots \ \lambda_{N-1}] \quad (9.28)$$

With these definitions, (9.26) may be applied N times, once for each column of Q , to give

$$\Phi_{ff}Q = Q\Lambda; \quad \text{or} \quad \Phi_{ff} = Q\Lambda Q^{-1} \quad (9.29)$$

This is called the *normal form* of the autocorrelation matrix Φ_{ff} , in which the eigenvalues appear explicitly. Furthermore, because (9.25) is true for each column of Q , and because the columns of Q are mutually orthogonal, it follows that

$$QQ' = I; \quad \text{that is, } Q' = Q^{-1} \quad (9.30)$$

and therefore, another expression of the normal form of Φ_{ff} is

$$\Phi_{ff} = Q\Lambda Q', \quad \text{or} \quad \Lambda = Q'\Phi_{ff}Q \quad (9.31)$$

The inverse of the normal form is easy to find, because Λ is diagonal, and so Λ^{-1} is also diagonal with elements $1/\lambda_n$; that is,

$$\Lambda^{-1} = \text{diag}[1/\lambda_0 \ 1/\lambda_1 \ \dots \ 1/\lambda_{N-1}] \quad (9.32)$$

Furthermore, for any two matrices A and B with inverses, $(AB)^{-1} = B^{-1}A^{-1}$. Therefore, using (9.29), the inverse of (9.31) is

$$\Phi_{ff}^{-1} = (Q')^{-1}(Q\Lambda)^{-1} = Q\Lambda^{-1}Q' \quad (9.33)$$

With the normal form and its inverse, we may now discuss the convergence of a steepest-descent algorithm starting with a revised version of the ideal weight recursion formula (9.9), which contains no information about the input correlation function, φ_{ff} :

$$b(k+1) = b(k) - \mu \nabla(b(k)) \quad (9.34)$$

First, we translate (9.34) using (9.17) so the optimal weight vector is $c_{\text{opt}} = 0$:

$$c(k+1) = c(k) - \mu \nabla_c(c(k)) \quad (9.35)$$

where ∇_c is the gradient vector translated so the gradient is zero at $c = 0$. Next, we substitute (9.7) for the *translated* gradient (noting that $\phi_{\text{pt}} = \Phi_{\text{ff}} b_{\text{opt}}$):

$$c(k+1) = c(k) - 2\mu\Phi_{\text{ff}}c(k) = (I - 2\mu\Phi_{\text{ff}})c(k) \quad (9.36)$$

Our next step is to *rotate* the c -coordinates to new coordinates, v , using the eigenvector matrix, as follows:

$$c = Qv, \text{ or } Q'c = v \quad (9.37)$$

The latter version follows from (9.30). Now, (9.36) becomes

$$Qv(k+1) = (I - 2\mu\Phi_{\text{ff}})Qv(k) = (Q - 2\mu\Phi_{\text{ff}}Q)v(k)$$

Multiplying by Q' and substituting (9.31), we have the normalized form of the ideal convergence formula:

$$\begin{aligned} v(k+1) &= (Q'Q - 2\mu Q'\Phi_{\text{ff}}Q)v(k) \\ &= (I - 2\mu\Lambda)v(k) \end{aligned} \quad (9.38)$$

In this normal form, because the off-diagonal elements of Λ are all zero, we now have N *independent* convergence formulas:

$$v_n(k+1) = (1 - 2\mu\lambda_n)v_n(k); \quad n = 0, 1, \dots, N-1 \quad (9.39)$$

Here, similar to (9.10) or (9.13), the geometric convergence ratio is $(1 - 2\mu\lambda_n)$, and so we see that the convergence criterion is now

$$\text{For steepest-descent convergence: } 0 < \mu < 1/\lambda_{\text{max}} \quad (9.40)$$

where λ_{max} is the maximum eigenvalue (corresponding with fastest convergence). Similarly, the time constants for convergence under ideal conditions, τ_w in (9.16) and τ_{MSE} in (9.22), may be modified as follows to reflect convergence along the principal axes:

$$\text{Steepest descent: } \tau_{wn} = \frac{1}{2\mu\lambda_n}; \quad \tau_{\text{MSE}n} = \frac{1}{4\mu\lambda_n}; \quad 0 \leq n < N \quad (9.41)$$

Note that the *slowest* descent (longest time constant) corresponds with the *smallest* eigenvalue, λ_{min} , which tends to dominate the latter part of the convergence process, as will be seen in examples that follow.

Concerning the range of μ in (9.40), we do not yet have a practical convergence criterion, because λ_{max} in (9.40) is known only if ϕ_{ff} is known. Hence, our

final step in the development of a steepest-descent algorithm is to express the *sum* of the eigenvalues in terms of a local estimate of signal power and use this in place of $\lambda_{\max}$ in (9.40) to provide an upper limit on μ .

Now the sum of the diagonal elements of any square matrix A is called the *trace* of A , or $\text{Tr}(A)$. Thus, the trace of Λ , which is a diagonal matrix, is the sum of the eigenvalues, and the trace of Φ_{ff} is the sum of the N diagonal elements; that is,

$$\text{Tr}(\Lambda) = \sum_{n=0}^{N-1} \lambda_n; \quad \text{Tr}(\Phi_{ff}) = \sum_{n=0}^{N-1} \phi_{ff}(0) = N\phi_{ff}(0) \quad (9.42)$$

These two traces are equal. The demonstration of equality can be made using an interesting property of the trace, namely, that the trace of the product of two square matrices is the trace of the reverse product; that is, if A and B are square matrices, then, using the definition of the product AB in (1.7) with the definition of the trace in (9.42),

$$\text{Tr}(AB) = \sum_{i=0}^{N-1} \sum_{n=0}^{N-1} a_{in} b_{ni} = \sum_{n=0}^{N-1} \sum_{i=0}^{N-1} b_{ni} a_{in} = \text{Tr}(BA) \quad (9.43)$$

The equality of $\text{Tr}(\Lambda)$ and $\text{Tr}(\Phi_{ff})$ follows when we use (9.31) in $\text{Tr}(\Lambda)$ and apply (9.43) along with (9.30):

$$\text{Tr}(\Lambda) = \text{Tr}(Q' \Phi_{ff} Q) = \text{Tr}(\Phi_{ff} Q' Q) = \text{Tr}(\Phi_{ff}) \quad (9.44)$$

Thus, the trace of Λ —the sum of the eigenvalues—is equal to the trace of Φ_{ff} —the sum of the diagonal elements of the autocorrelation matrix, each of which is $\phi_{ff}(0)$ as in (8.6); that is,

$$\text{Tr}(\Lambda) = \sum_{n=0}^{N-1} \lambda_n = N\phi_{ff}(0) = N\sigma_f^2 \quad (9.45)$$

Here, we use σ_f^2 to represent the variance, or average power, of the signal f , which is the same as $\phi_{ff}(0)$ in (7.30). If σ_f^2 changes during the adaptive process, then it must be adjusted along with the weights. But at least $\sigma_f^2(k)$ is easy to estimate locally in time as the average squared signal amplitude (one estimation method is described in the next section), and so we now have a practical version of the convergence criterion in (9.40) for steepest-descent algorithms. To be consistent with the ideal weight convergence described by (9.9), we set

$$\mu = u/N\sigma_f^2(k) \quad (9.46)$$

Then, the steepest-descent algorithm in (9.34) and the convergence criterion in (9.40) produce the following result:

$$\text{Steepest descent: } b(k+1) = b(k) - \frac{u}{N\sigma_f^2(k)} \nabla(k); \quad 0 < u < 1 \quad (9.47)$$

With this result, we are ready to derive the *LMS algorithm*, which was first published by Widrow and Hoff¹ and has since become the most widely applied algorithm in adaptive signal processing. All we need is a local estimate of the gradient vector, which the LMS algorithm supplies. We recall that the ideal gradient in (9.5) and (9.9) is the derivative of the MSE with respect to the weight vector, b . In the Widrow LMS algorithm, the local estimate of the MSE is the squared error, e_k^2 . First, we define the *input signal vector* of length N , consisting of the most recent N samples of the input signal, f , in *reverse order*:

$$f(k) = [f_k \ f_{k-1} \ \dots \ f_{k-N+1}]' \quad (9.48)$$

Using (8.1) and (8.3) in Chapter 8, with $b(k)$ as the current weight vector, our expression for e_k is

$$e_k = d_k - b(k)'f(k) \quad (9.49)$$

(Remember here that *subscript* k denotes the k th sample, and *argument* k denotes the k th vector.) Using (9.49), Widrow's estimate of the gradient vector for use in the LMS algorithm is

$$\hat{\nabla}(k) = \frac{\partial e_k^2}{\partial b} = 2e_k \frac{\partial e_k}{\partial b} = -2e_k f(k) \quad (9.50)$$

Using this gradient estimate in place of $\nabla(k)$ in (9.47) produces the LMS algorithm:

$$\text{LMS algorithm: } b(k+1) = b(k) + \frac{2u}{N\sigma_f^2(k)} e_k f(k); \quad 0 < u \ll 1 \quad (9.51)$$

The condition $u \ll 1$ is in (9.51) to emphasize that u is small in practice (sometimes on the order of 0.1 and often much less) depending on how fast the signal environment is changing. The LMS gradient estimate causes the weight vector to converge on a noisy version of the steepest descent path, as we will see in the first example in the following section.

Convergence of the LMS weights and mean squared error may be approximated by the steepest-descent time constants in (9.41), with μ specified in (9.46).

In general, as stated previously, the convergence of the LMS algorithm is governed increasingly as the MSE approaches $\text{MSE}_{\min}$ by the minimum eigenvalue, $\lambda_{\min}$, which corresponds with the longest convergence time constant. For a conservative description of LMS convergence, we substitute $\lambda_{\min}$ for λ_n and (9.46) for μ into (9.41) and obtain these LMS convergence time constants:

$$\text{LMS Convergence: } \tau_w = \frac{N\sigma_f^2}{2u\lambda_{\min}}; \quad \tau_{\text{MSE}} = \frac{N\sigma_f^2}{4u\lambda_{\min}}; \quad 0 < u \ll 1 \quad (9.52)$$

Thus, convergence of the LMS algorithm is slower than ideal convergence when $\lambda_{\min}$ is less than the average eigenvalue. If the adaptive system is designed with $\lambda_{\min}$ unknown, which is usually true, one must plan for the worst case. For example, suppose the input to an adaptive filter with two weights has a sinusoidal component at ν Hz-s. Then, it is easy to show (Exercise 9.13) that the ratio of $\lambda_{\min}$ to the average eigenvalue is

$$\frac{\lambda_{\min}}{\lambda_{\text{av}}} = 1 - \cos(2\pi\nu)$$

Thus, other factors being equal, convergence times for the LMS algorithm are longer at lower frequencies.

9.5 LMS Examples

Our first example of adaptive weight convergence using the LMS algorithm is illustrated in Figure 9.4, which repeats Figure 9.2 for one-step prediction of a sine wave with two weights and ideal convergence, and also includes the weight track for LMS convergence. The adaptive gain is $u = 0.05$ for both weight tracks; however, convergence requires about 50 iterations on the ideal track and 350 iterations on the LMS track. On the LMS track, we can observe erratic direction and step size due to inaccuracy in the gradient estimate (9.50) and the steepest-descent weight track, which is orthogonal to the MSE contours.

A plot of e_k^2 vs. k in this example is shown in Figure 9.5. The plot is equivalent to the *learning curve* described in Section 9.3, which is a plot of $E[e_k^2]$ vs. k . It covers 51 iterations of the LMS algorithm and is seen to converge toward the minimum MSE (zero) over this range. Convergence is comparable to the ideal convergence plotted in Figure 9.3. The latter is plotted as a dashed line for comparison with LMS convergence in this example.

The error signals used for the LMS track in Figure 9.4 and the learning curve in Figure 9.5 were generated using a MATLAB function supplied with

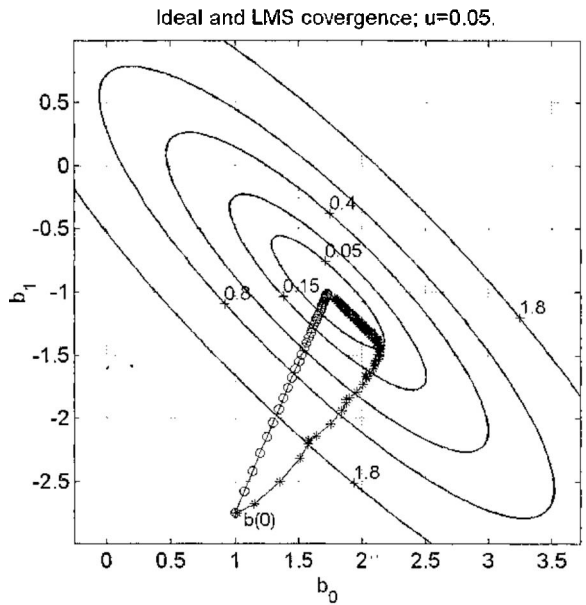


FIGURE 9.4
Comparison of ideal and LMS weight convergence for the predictor with two weights (Figure 9.2). The number of iterations is $K = 50$ for ideal convergence and 350 for LMS convergence.

this text called *lms_filter*:

$$[g, b] = \text{lms_filter}(b, u, d, f)$$

Referring to Figure 9.1, this function produces vector g as the result of filtering f and adapting at each step using the error, $e_k = d_k - g_k$. The weight vector $b(k)$ is updated and stored at each step, so that the entire weight track is stored as an output. This is a simple implementation of the LMS algorithm in which σ_k^2 is held fixed. It is useful for producing results like those in Figures 9.4 and 9.5 but may need to be modified for other uses.

Our next example is an illustration of why adaptive signal processing may be preferred with nonstationary signals. We will keep the same, simple two-weight predictor described in Chapter 8, Section 8.4, and illustrated in Figure 8.7. To have a simple nonstationary signal, we use a single “frame” of signal d_k , which is shown in the upper plot in Figure 9.6. The frame consists of 1000 samples of a unit sine wave with frequency changing from $\nu = 0.05$ to $\nu = 0.10$ Hz-s at sample $k = 750$. We assume that, prior to the time of this frame, the frequency of d_k has been constant at $\nu = 0.05$ Hz-s. Now, suppose we are using a two-weight predictor to compress this signal and wish, therefore, to produce a small residue, e_k . If fixed optimal weights are computed using the

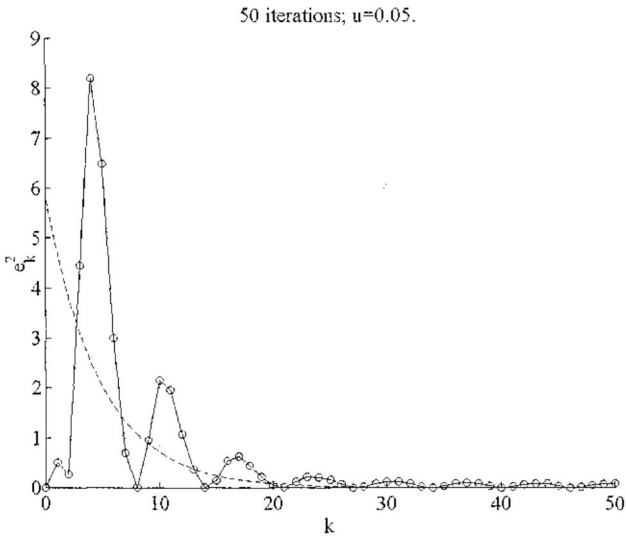


FIGURE 9.5
Comparison of convergence of e_k^2 (connected points) using the LMS algorithm with ideal convergence of the MSE (dashed line) for the prediction example in Figure 9.3. The adaptive gain is $\mu = 0.05$ in both cases.

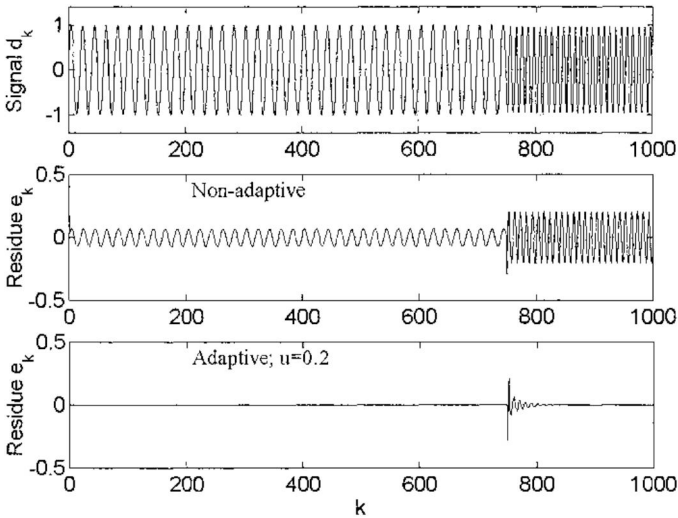


FIGURE 9.6
Nonadaptive and adaptive signal compression using a one-step predictor with two weights. The frequency of the signal, d_k , doubles at $k = 750$. The fixed (nonadaptive) predictor has weights optimized for the entire frame. The adaptive predictor has adapted previously to the first frequency, so its average prediction error (residue) is much smaller.

method of Chapter 8, the residue, shown in the center plot in Figure 9.6, cannot be reduced to zero due to the frequency change in d_k . On the other hand, if an LMS *adaptive predictor* has previously adapted to the initial sine wave and again adapts to the frequency change in this frame, the residue remains close to zero over most of the frame, as shown in the lower plot in Figure 9.6. The only noticeable residue is where the change occurs at $k = 750$. In this particular example, the nonadaptive total squared error (TSE) is 7.6, and the adaptive TSE is only 0.2, indicating a much smaller average residue and, hence, significant compression of the signal.

If an adaptive filter is used in signal compression, the question of recovery arises. Can the original signal be recovered from the residue? The answer is yes, and with no additional information except the adaptive gain constant, u . Recovery proceeds as described in Chapter 4, (4.32), and as discussed in Chapter 8, Section 8.10. The only difference is that the LMS algorithm must be applied at each step to adjust the weight vector during recovery. It is easy to see in the formulation (9.51) of the LMS algorithm that the adjustment is realizable, because at each step, the weights for the *next* step are computed in terms of quantities that are available during recovery.

9.6 Direct Descent and the RLS Algorithm

Comparing the steepest-descent formula (9.47) with the ideal formula (9.9), and comparing the weight tracks in Figure 9.4, we may note that the presence of Φ_{ff}^{-1} in (9.9) has the effect of steering the weights away from the direction of the gradient to a direct path leading to the minimum MSE. Thus, it is reasonable to look for a formula like (9.51) that, in addition to estimating the gradient at each step, also estimates Φ_{ff}^{-1} at each step. The *Recursive Least Squares* (RLS) algorithm is such a formula.

The RLS algorithm exists in different forms.^{4,6,45} The version we use here is based on an approximation to (9.9) of the form

$$b(k+1) = b(k) - u\hat{\Phi}_{ff}^{-1}\hat{\nabla}(k)$$

The simplest local gradient estimate is the LMS gradient estimate (9.50). Substituting this estimate for $\hat{\nabla}(k)$ and using $\hat{\Phi}_{ff}^{-1}(k)$ for the current estimate of $\hat{\Phi}_{ff}^{-1}$, we have

$$b(k+1) = b(k) + 2u\hat{\Phi}_{ff}^{-1}(k)e_k f(k) \quad (9.53)$$

Let us now consider the problem of estimating the inverse autocorrelation matrix, Φ_{ff}^{-1} . The information in Φ_{ff} is contained in N correlation elements, $\varphi_{ff}(0:N-1)$. With the exception of $\varphi_{ff}(0)$, which is the input power σ_f^2 , a

correlation value $\varphi_{ff}(n) = E[f_k f_{k-n}]$ cannot be estimated without using a history of at least n samples. Thus, the simplest local estimate of $\varphi_{ff}(n)$ is the sample product $f_k f_{k-n}$, and, using the definition of the k th signal vector, $f(k)$ in (9.48), it follows that the simplest local estimate of $\hat{\Phi}_{ff}$ at the k th step is

$$\hat{\Phi}_{ff}(k) = f(k)f'(k) \quad (9.54)$$

In Chapter 8, where stationary signals were assumed, we used the available history, that is, the entire signal vector f , to estimate $\varphi_{ff}(n)$ as the average (or total) product. If f is nonstationary such that φ_{ff} changes slowly, then it is reasonable to use only the recent history of f to estimate φ_{ff} . If the signal $f(t)$ is locally periodic, it is reasonable to average over one or more periods. If the signal statistics are changing slowly with time, we use the following modification of (9.54), which gradually "forgets" the past history of the estimate:

$$\hat{\Phi}_{ff}(k) = (1 - \alpha)\hat{\Phi}_{ff}(k-1) + \alpha f(k)f'(k); \quad 0 < \alpha < 1 \quad (9.55)$$

When α is close to zero, current events in $f(t)$ have little effect on the estimate, and conversely, when α is close to one, current events in $f(t)$ have a large effect on the estimate.

The recursive estimate (9.55) produces the correlation estimate as essentially a lowpass-filtered version of the instantaneous estimate, $f(k)f'(k)$. The lowpass transfer function is seen to be

$$H(z) = \frac{\alpha}{1 - (1 - \alpha)z^{-1}} \quad (9.56)$$

The corresponding impulse response is

$$h_k = \alpha(1 - \alpha)^k; \quad k \geq 0 \quad (9.57)$$

and the correlation estimate is the convolution of h_k with $f(k)f'(k)$; that is,

$$\begin{aligned} \hat{\Phi}_{ff}(k) &= \sum_{n=0}^k h_n f(k-n)f'(k-n) \\ &= \alpha \sum_{n=0}^k (1 - \alpha)^n f(k-n)f'(k-n) \end{aligned} \quad (9.58)$$

Here again, as in (9.13), we can see that when α is small, there is an exponential decay in the memory of the past correlation estimate, and the "forgetting

time constant'' is

Time constant for forgetting the correlation estimate:
 $\tau_{\hat{\Phi}_{ff}} \approx \frac{1}{\alpha} \text{ iterations; } 0 < \alpha \ll 1$

(9.59)

To eliminate the need for a matrix inversion at each step in (9.53), we now proceed to change (9.55) from a recursive estimate of $\hat{\Phi}_{ff}$, to a recursive estimate of $\hat{\Phi}_{ff}^{-1}$. First, we pre-multiply (9.55) by $\hat{\Phi}_{ff}^{-1}(k)$ and post-multiply by $\hat{\Phi}_{ff}^{-1}(k-1)$ to obtain

$$\hat{\Phi}_{ff}^{-1}(k-1) = (1-\alpha)\hat{\Phi}_{ff}^{-1}(k) + \alpha\hat{\Phi}_{ff}^{-1}(k)f(k)f'(k)\hat{\Phi}_{ff}^{-1}(k-1) \quad (9.60)$$

Next we post-multiply this result by $f(k)$:

$$\begin{aligned} \hat{\Phi}_{ff}^{-1}(k-1)f(k) &= (1-\alpha)\hat{\Phi}_{ff}^{-1}(k)f(k) + \alpha\hat{\Phi}_{ff}^{-1}(k)f(k)f'(k)\hat{\Phi}_{ff}^{-1}(k-1)f(k) \\ &= \hat{\Phi}_{ff}^{-1}(k)f(k)(1-\alpha + \alpha f'(k)\hat{\Phi}_{ff}^{-1}(k-1)f(k)) \end{aligned} \quad (9.61)$$

To make this less complicated, we define

$$S(k) = \hat{\Phi}_{ff}^{-1}(k-1)f(k); \quad S'(k) = f'(k)\hat{\Phi}_{ff}^{-1}(k-1) \quad (9.62)$$

(The second form follows because $\hat{\Phi}_{ff}^{-1}$ is symmetric and equal to its transpose.) Now (9.61) becomes

$$S(k) = \hat{\Phi}_{ff}^{-1}(k)f(k)(1-\alpha + \alpha f'(k)S(k)) \quad (9.63)$$

Note that the factor in parentheses here is a scalar. Dividing by this factor and post-multiplying by $S'(k)$ in (9.62), we have

$$\hat{\Phi}_{ff}^{-1}(k)f(k)f'(k)\hat{\Phi}_{ff}^{-1}(k-1) = \frac{S(k)S'(k)}{1-\alpha + \alpha f'(k)S(k)} \quad (9.64)$$

Our final step is to substitute α times this result for the last term in (9.60) and rearrange the terms to obtain

$$\hat{\Phi}_{ff}^{-1}(k) = \frac{1}{1-\alpha} \left(\hat{\Phi}_{ff}^{-1}(k-1) - \frac{\alpha S(k)S'(k)}{1-\alpha + \alpha f'(k)S(k)} \right) \quad (9.65)$$

In this result, with the definition of $S(k)$ in (9.62), we have an iterative algorithm for computing $\hat{\Phi}_{ff}^{-1}(k)$ at each step k . It is reasonable also to use

the same smoothed estimate of total power, using either $\varphi_H(0)$ or the instantaneous squared value, f_k^2 , as the input to $H(z)$ in (9.56). In either case, the equivalent of (9.55) for the power estimate is

$$\sigma_f^2(k) = (1 - \alpha)\sigma_f^2(k-1) + \alpha f_k^2 \quad (9.66)$$

Note that we could, if desired, replace f_k^2 with the average diagonal element of $f(k)f'(k)$ as a current estimate of $\varphi_H(0)$.

Initial conditions must be specified for the functions in both of these recursive estimation formulas. For $\sigma_f^2(0)$ in (9.66), we assign a value, say σ_{f0}^2 , as an a priori estimate of the input signal power. Then σ_{f0}^2 is also the a priori estimate of $\varphi_H(0)$ and, in the absence of any other correlation information, we may use $\sigma_{f0}^2 I$ as the initial estimate of Φ_H . Thus, the initial conditions for (9.65) and (9.66) are

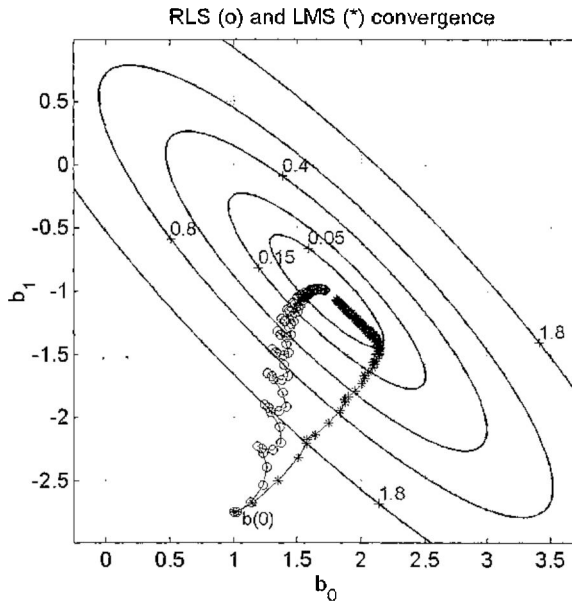
$$\text{Initial conditions: } \begin{cases} \sigma_f^2(0) = \sigma_{f0}^2 \\ \Phi_H^{-1}(0) = \frac{1}{\sigma_{f0}^2} I \end{cases} \quad (9.67)$$

With these initial conditions, we also require starting values $b(0)$ and $f(0)$ for the weight vector and signal vector, respectively, as we required with the LMS algorithm. After startup, the RLS algorithm then consists of (9.62), (9.65), and (9.53) and may be summarized as follows:

RLS Algorithm: $S(k) = \hat{\Phi}_H^{-1}(k-1)f(k)$ $\hat{\Phi}_H^{-1}(k) = \frac{1}{1-\alpha} \left(\hat{\Phi}_H^{-1}(k-1) - \frac{\alpha S(k)S'(k)}{1-\alpha + \alpha f'(k)S(k)} \right); \quad 0 < \alpha < 1$ $e_k = d_k - b'(k)f(k)$ $b(k+1) = b(k) + 2u\hat{\Phi}_H^{-1}(k)e_k f(k); \quad 0 < u < 1$	(9.68)
--	--------

Compared with the LMS algorithm (9.51), the RLS algorithm requires much more computation at each step and also requires the specification of two parameters, α and u . However, it has the potential, provided $\hat{\Phi}_H^{-1}(k)$ is a good estimate of the inverse autocorrelation matrix, of producing faster convergence to b_{opt} , the optimal weight vector.

An example of RLS weight convergence is provided in Figure 9.7, which is like Figure 9.4 with the RLS weight track in place of the ideal weight track. Compared with the ideal and LMS tracks in Figure 9.4, the RLS track is more like the ideal track. It follows a more or less direct path to the optimal point, which, we recall, is at $[b_0, b_1] = [\sqrt{3}, -1]$ in this example. The LMS track

**FIGURE 9.7**

Comparison of RLS and LMS weight convergence for the predictor with two weights (Figure 9.2). The number of iterations was $K = 150$ for RLS convergence and 300 for LMS convergence. RLS parameters were $[u, \alpha] = [0.02, 0.2]$. LMS adaptive gain was $\mu = 0.05$.

contains twice as many iterations, and, following the path of steepest descent, converges slowly as the gradient approaches zero.

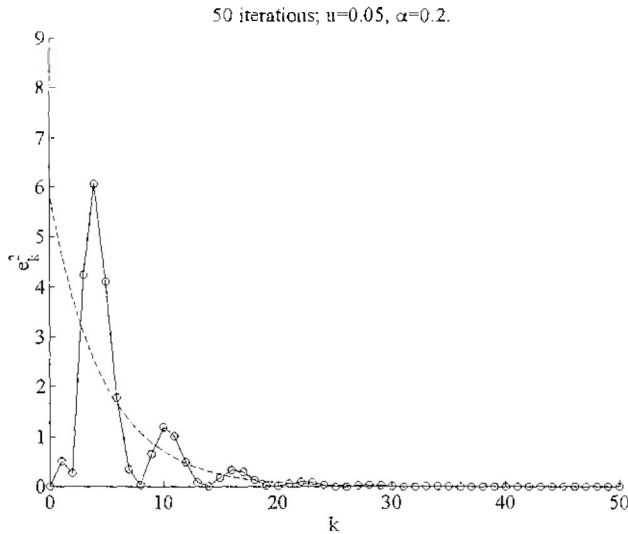
A learning curve for the RLS algorithm is shown in Figure 9.8. The adaptive gain was set to $\mu = 0.05$ so the result could be compared with the LMS learning curve in Figure 9.5, and we can see that convergence is improved with the RLS algorithm in this example.

As in the case of Figures 9.4 and 9.5, the plots in Figures 9.7 and 9.8 were made using a MATLAB function supplied with this text. The function is called *rls_filter* and has arguments as follows:

$$[g, b] = \text{rls_filter}(b, u, \alpha, d, f)$$

This function is the same as the *lms_filter* function described previously, except that it implements the RLS algorithm, and its arguments include the forgetting factor, α , used to adjust the correlation and power estimates.

Having seen these examples and having discussed and compared the LMS and RLS adaptive algorithms, the reader may wonder why, if one or the other of the algorithms does not converge as fast as desired, the adaptive gain cannot be increased until it does so. The answer is that the behavior of

**FIGURE 9.8**

Comparison of convergence of e_k^2 (connected points) using the RLS algorithm with ideal convergence of the MSE (dashed line) for the prediction example in Figure 9.3. The adaptive gain is $u = 0.05$ in both cases.

the system *after* convergence may suffer if u is too close to its stability limit (one). After convergence under stationary conditions, the weight vector, $b(k)$, continues to “adapt,” and wanders around under the error surface in the vicinity of its optimum, b_{opt} . That is, due to inaccuracies in local estimates, the MSE climbs up the bowl-shaped quadratic error surface and is then driven back down toward MSE_{min} . The continuation of this process results in a deviation of the MSE from its minimum value, MSE_{min} , which is undesirable. Thus, there is a tradeoff between convergence speed and convergence accuracy, which we discuss in the next section.

9.7 Measures of Adaptive System Performance

The two principal measures of adaptive system performance are those we discussed in the preceding sections. First are the time constants for convergence of the weights and the MSE, which measure the speed of adaptation. Second is a measure called *misadjustment*, which is a normalized measure of the average deviation of the MSE from its minimum value.

For convergence time constants, we have the weight and MSE time constants, τ_w and τ_{MSE} , which we derived in Section 9.3, (9.16) and (9.22), and in

Section 9.4, (9.52):

Convergence time constants; $0 < u \ll 1$:	
Ideal conditions: $\tau_w \approx \frac{1}{2u}$; $\tau_{\text{MSE}} \approx \frac{1}{4u}$ iterations;	(9.69)
Steepest descent: $\tau_w \approx \frac{N\sigma_f^2}{2u\lambda_{\min}}$; $\tau_{\text{MSE}} \approx \frac{N\sigma_f^2}{4u\lambda_{\min}}$ iterations	

Because these time constants are inversely proportional to u , why not increase u toward one and have a more responsive, faster-converging adaptive system? As we have seen, the answer is that in practical systems with noisy estimates of the gradient and/or the autocorrelation matrix, increasing u means increasing the *excess MSE* after convergence. The excess MSE is the increase in the MSE above $\text{MSE}_{\min}$ after convergence due to these noisy estimates. From (9.19), the excess MSE may be written as

$$\begin{aligned} \text{Excess MSE} &= \text{MSE} - \text{MSE}_{\min} \text{ after convergence} \\ &= E[c' \Phi_{ff} c] \end{aligned} \quad (9.70)$$

For example, the noisy LMS gradient estimate in (9.50) causes the weight vector to be adjusted in the vicinity of $\text{MSE}_{\min}$ at each iteration after convergence, and thus contributes to the excess MSE through variations in the translated weight vector, c , in accordance with (9.70).

To derive a simple and practical estimation of the excess MSE for the LMS algorithm, it is necessary to assume that these variations in c consist of small, independent jumps away from the optimal point at $c = 0$. With this approximation, under certain conditions, the excess MSE is

$$\text{Excess MSE} \approx uN \text{MSE}_{\min}; \quad 0 < u \ll 1 \quad (9.71)$$

The conditions are that u must be small, the eigenvalues of Φ_{ff} must be approximately equal, and the excess MSE is small (on the order of 0.1 or less). The derivation of this useful approximation, which is lengthy and involves a number of assumptions that must be supported, is not included here. Derivations are provided in Widrow et al.¹⁸ and in Chapters 5 and 6 of Widrow and Stearns.⁵

Instead of the excess MSE, we use a normalized measure of the excess MSE called *misadjustment* (M), defined as follows:

Misadjustment: $M \equiv \frac{\text{Excess MSE}}{\text{MSE}_{\min}} \approx uN$	(9.72)
--	--------

The misadjustment and the convergence time constants (9.69) constitute the principal performance measures for adaptive system performance.

The approximation in (9.72) is said to be good for values of M up to 0.1⁴⁴. This assumption is investigated in Exercises 15 and 16. Both exercises

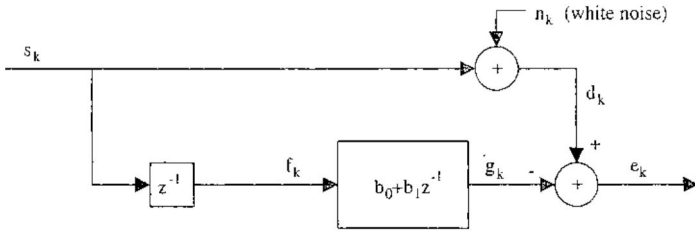


FIGURE 9.9

White noise (n_k) is added to d_k in the adaptive predictor used in previous examples in order to produce a nonzero minimum MSE.

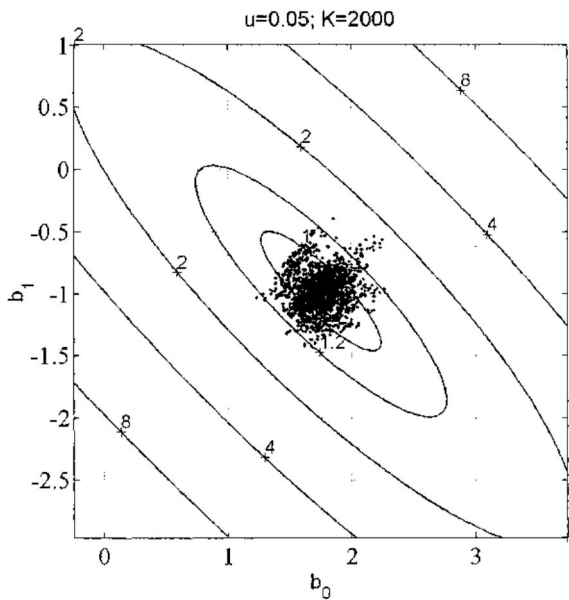


FIGURE 9.10

Result of Exercise 15, illustrating how the LMS algorithm drives the converged weight vector in the vicinity of the minimum MSE and causes misadjustment.

investigate (9.72) using the example we have been using throughout this chapter, which is the one-step predictor in Figure 8.7 with two weights and a sinusoidal input. The example is modified as shown in Figure 9.9 by adding white noise to d_k in order to produce a nonzero minimum MSE. Notice that the adaptive filter can cancel the sine wave but is not able to cancel the noise, because the noise does not appear in f_k .

Exercise 15 consists of starting the weight vector at b_{opt} and running 2000 iterations of the LMS algorithm with adaptive gain $u = 0.05$. The result is illustrated in Figure 9.10, where 2000 iterations of the weight vector are

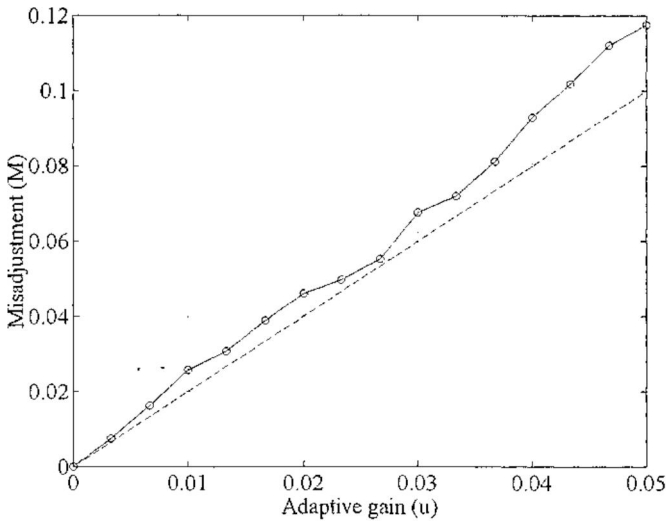


FIGURE 9.11

Result of Exercise 16, in which misadjustment is measured experimentally under stationary conditions with different adaptive gain values. The dashed line represents the theoretical approximation in (9.72).

plotted as points on the weight plane. The plot illustrates the typical random excursions of the converged weight vector in the vicinity of the minimum MSE, which are the cause of misadjustment.

Exercise 16 consists of starting the weight vector at b_{opt} and making several independent runs of the LMS algorithm for each of several values of u , measuring the misadjustment in each case by averaging the squared prediction error, e_k^2 . The result is plotted in Figure 9.11, which shows the experimental points with the approximation in (9.72). Even though the eigenvalues of Φ_{ff} , which are $[0.1340, 1.8660]$ in this example, are quite different and do not fulfill the second condition for the validity of (9.72), the result verifies (9.72) as a useful approximation. However, as the misadjustment increases above 0.1, M becomes increasingly larger than the approximation in (9.72).

If we assume the validity of (9.72) as well as the ideal time constants (9.69), which are only approximations for LMS and RLS convergence as we have seen, we may combine the formulas for M and τ_{MSE} to obtain

$$M \approx \frac{N}{4\tau_{\text{MSE}}}; \quad 0 < u < 1, \quad 1 < M < 0.1 \quad (9.73)$$

Although this formula does not apply in all situations, it nevertheless serves to summarize adaptive system performance. It demonstrates that, in adaptive signal processing, low misadjustment and fast convergence are desirable but conflicting requirements. Faster convergence means noisier performance with

a greater excess MSE. In fact, noting that approximately four time constants ($4\tau_{\text{MSE}}$) are required for convergence from an arbitrary MSE to MSE_{min} , we can restate (9.73) as follows,¹⁸

$$\text{Misadjustment } (M) \approx \frac{\text{Number of weights}(N)}{\text{Convergence time}(4\tau_{\text{MSE}})}; \quad \begin{cases} 0 < u < 1 \\ 0 < M < 0.1 \end{cases} \quad (9.74)$$

This is perhaps the simplest and most general statement one can make about the performance of linear adaptive systems that use algorithms like those we have been discussing in this chapter.

Beyond these considerations, at least at the time of this writing, the design of adaptive systems is less than an exact science. The number of weights (N), the adaptive gain (u), and the forgetting factor (α) must be chosen with the performance requirements in mind, but once they are chosen, performance in nonstationary conditions also is affected by unpredictable changes in signal statistics. The fact that adaptive signal processing systems work and survive in so many different applications, like the fact that biological systems do the same, is quite remarkable.

9.8 Other Adaptive Structures and Algorithms

In this chapter, we introduced adaptive signal processing using an FIR filter as the adaptive filter and discussed the LMS and RLS adaptive algorithms. These today are the simplest and most widely applicable components of adaptive systems, but there are other kinds of components and applications discussed in the literature.

One type of application using the FIR filter we have not mentioned is the *adaptive array*,^{3,5,16} a form of which is illustrated in Figure 9.12. (A simple form

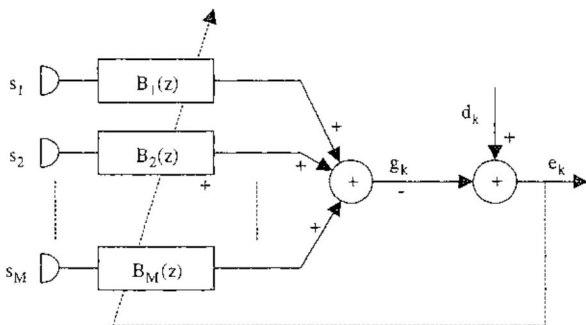


FIGURE 9.12

An adaptive array, consisting of a multiple-input adaptive system. To achieve a common goal, the adaptive filters are adjusted using a single error, e_k .

of this was used in Chapter 8, Exercise 24.) The array consists of M sensors, s_1 through s_M , each s_m feeding its adaptive filter, $B_m(z)$. The desired signal, d_k , is derived in a manner that points the array at a signal source, or away from an interfering source, or in some other manner, in a process known as *adaptive beamforming*.^{3,5} Systems like this, with more than one adaptive filter driven by a single error, are known as *multiple-input* adaptive systems.

When each of the adaptive filters in Figure 9.12 consists of just a single weight, the system is called an *adaptive linear combiner*.^{5,6} The adaptive linear combiner adapts in the same manner as the FIR filter, except that the elements of the input vector f are available simultaneously, that is, at the same time step. For example, the adaptive linear combiner could take the form of a linear array of photo sensors that moves relative to (scans) a visual image, or it could take the form of a planar array of sensors viewing a changing scene.

Other types of adaptive filter structures have been proposed. One is the *adaptive lattice*,^{33–35,38,39,44} which is developed by allowing the weights, κ and λ , of one of the lattice structures described in Chapter 4, Section 11, to be adapted with a version of the LMS algorithm.

Recursive filters as well as nonlinear filter structures have been applied in adaptive signal processing.^{5,6,21–23} Some of the nonlinear structures, for example, use *neural networks* consisting of networks of threshold elements that act like neurons. Recursive adaptive filters have not found a large number of applications, because, as discussed previously, the associated MSE functions are generally not quadratic and may contain local minima. But, recursive filters have poles, and their operations are, in general, reversible, so they remain candidates for adaptive systems in compression applications and system identification.

Other adaptive algorithms include special algorithms for adapting recursive and nonlinear systems,⁶ including *genetic* algorithms,^{5,25} which use the evolutionary or “survival of the fittest” approach to minimize the MSE.

There is also a class of algorithms known as *blind* algorithms used in communications systems that adapt to changing signal or channel conditions. In *blind equalization*,⁶ the desired signal, d , which is known at the source but not at the receiver, is approximated (guessed) at the receiver using portions or properties of the received signal, and is then used to modify the received signal. Blind equalization is used in adaptive communications receivers to reduce channel distortion as well as interfering noise and other signals, including echos of the desired signal.

9.9 Exercises

1. In Section 3, the weight convergence formula (9.12) was proved by taking the z-transform of the recursion formula (9.10). Instead of using the z-transform, prove (9.12) by induction.

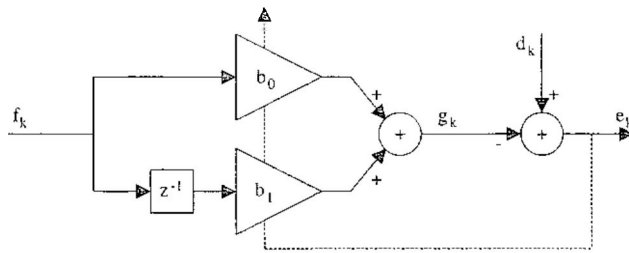


FIGURE 9.13
Adaptive filter with two weights.

2. Examine the validity of the weight-convergence time constant (9.16) by making four subplots in the same figure with $u = 0.05$ in the first plot, and then 0.1, 0.2, and 0.4 in the rest of the subplots. In each subplot, complete the following:
 - a. Plot $1 - e^{-k/\tau_w}$ vs. k for $k = [0, 5\tau_w]$ using a dashed line.
 - b. Plot symbols at $k = \tau_w, 2\tau_w, \dots, 5\tau_w$.
 - c. Plot the exact convergence, $1 - (1 - 2u)^k$ as a connected discrete plot. Comment on the validity of (9.16) with respect to each plot.
3. A simple adaptive filter with two weights is illustrated in Figure 9.13. The signals f and d are:

$$f_k = \sin(2\pi k/N); \quad d_k = 2\cos(2\pi k/N)$$

- a. Express the autocorrelation matrix Φ_{ff} as a function of N , the number of samples per cycle.
 - b. Express the cross-correlation vector, ϕ_{fd} , as a function of N .
 - c. Express the mean squared error (MSE) as a quadratic function of b_0 and b_1 .
4. For the adaptive filter described in Exercise 3, complete the following:
 - a. Write an expression for the gradient of the error surface.
 - b. Write an expression for the optimal weight vector.
5. For the adaptive filter described in Exercise 3, assume $N = 10$ and complete the following:
 - a. Calculate the optimal weight vector, b_{opt} .
 - b. On the $b_0 - b_1$ plane, plot contours (curves of constant MSE) for $N = 10$ and the following MSE values: 0.0001, 1.0, 5.0, and 10.0. Use $b \pm 6$ to define the limits of the b -plane. (Note: This plot is most easily done using the MATLAB *contour* function.)

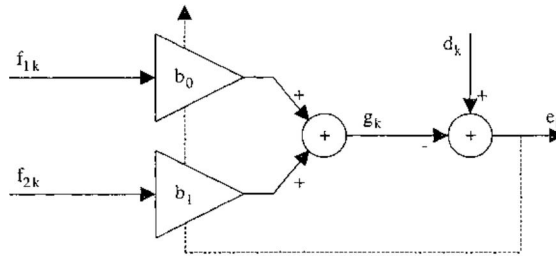


FIGURE 9.14
Adaptive linear combiner.

6. Suppose we modify Figure 9.13 to produce the *adaptive linear combiner* in Figure 9.14. Now there are two input signals instead of one, and no delays are involved. We assume stationary signals with the following statistics:

$$\begin{aligned} E[d_k^2] &= 36, \quad E[f_{1k}^2] = 2, \quad E[f_{2k}^2] = 3, \quad E[f_{1k} f_{2k}] = 1, \\ E[f_{1k} d_k] &= 6, \quad E[f_{2k} d_k] = 4 \end{aligned}$$

- a. Express Φ_{ff} and ϕ_{fd} for this case.
 - b. Write an expression for MSE as a function of b_0 and b_1 .
 - c. Find the optimal weight vector, b_{opt} .
 - d. Determine the minimum MSE.
7. For the adaptive linear combiner in Figure 9.14,
- a. Find the eigenvalues, λ_0 and λ_1 .
 - b. Find the eigenvector matrix, Q .
 - c. Plot MSE contours on the $b_0 - b_1$ plane using MSE values [10, 50, 100, 200].
 - d. In the same figure, plot translated (c) axes and rotated (v) axes. Note that the v axes are the principal axes of the elliptical MSE contours and thus indicate the steepest and least-steep descent paths on the MSE surface.
8. For the adaptive linear combiner in Figure 9.14,
- a. Write an expression for the gradient vector as a function of the weight vector.
 - b. Starting with initial weight vector $b(0) = [10; 10]$ and using parameters $\mu = 0.1$ and σ_f^2 equal the average input signal power in Exercise 6, compute the first five weight values on a path that converges directly to b_{opt} .
 - c. Repeat part (b) using the path of steepest descent instead of the direct path.

9. For the adaptive linear combiner in Exercise 3, complete the following:
 - a. Write an expression for the gradient vector as a function of the weight vector.
 - b. Starting with initial weight vector $b(0) = [2; -8]$ and using parameters $\mu = 0.1$ and σ_f^2 equal $\phi_{ff}(0,0)$ compute the first five weight values on a path that converges directly to b_{opt} .
 - c. Repeat part (b) using the path of steepest descent instead of the direct path.
10. In the prediction example in Section 9.5, Figure 9.6, the nonstationary input is described as a unit sine wave that is stationary at 0.05 Hz-s for a long time and then doubles in frequency.
 - a. What is the optimal weight vector, b_{opt} , for predicting the initial part of the signal?
 - b. What is the minimum MSE associated with (a)?
 - c. In the upper of three subplots, plot the complete data frame as it appears in the upper plot in Figure 9.6.
 - d. Operate the LMS algorithm on the data frame with 1000 samples with b initialized to b_{opt} and $\mu = 0.2$. Plot the residue after $k = 0$ in the center subplot and observe that your plot looks the same as the lower plot in Figure 9.6.
 - e. Operate the RLS algorithm on the data frame with 1000 samples with b initialized to b_{opt} , $\mu = 0.2$, and $\alpha = 0.001$. Plot the residue after $k = 0$ in the lower subplot, and compare the result with the center plot.
11. An adaptive filter with two weights is used as in Figure 9.15 to minimize the squared error, e_k^2 , with the following inputs at frequency $\nu = 0.1$ Hz-s:

$$f_k = \cos(2\pi\nu k); \quad d_k = 3\cos(2\pi\nu k + \pi/5); \quad 0 \leq k < \infty$$
 - a. Find the autocorrelation matrix Φ_{ff} .
 - b. Find the eigenvalues of Φ_{ff} by hand using (9.24).

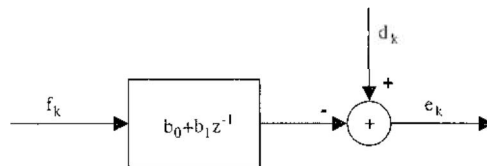


FIGURE 9.15
Adaptive filter with two weights.

- c. Find the time constant for convergence of the mean squared error using the LMS algorithm with $u = 0.1$.
 - d. Plot a learning curve (MSE vs. k) for the LMS algorithm that begins at the mean squared value of d_k and converges to zero over $k = 0:99$.
 - e. On the same graph, plot a learning curve using the average eigenvalue in place of $\lambda_{\min}$.
 - f. Run the LMS algorithm with signals d and f , beginning with weight vector $b(0) = [0, 0]'$, and plot e_k^2 vs. k , again on the same graph. Note how a running average of this plot would run between the two theoretical curves.
12. Repeat Exercise 11, but this time, use the RLS algorithm with $u = \alpha = 0.1$ and $b(0) = [0, 0]'$. Plot theoretical learning curves for ideal convergence and convergence using the LMS algorithm. This time, use the range $k = 0:50$. Note the faster convergence of the RLS algorithm, and comment on the comparison of the squared error plot with the learning curves.
 13. Suppose, as in the two preceding exercises, that the input to an adaptive filter with two weights is an arbitrary sinusoid at frequency ν Hz-s, say

$$f_k = A \cos(2\pi\nu k + \gamma); \quad 0 \leq k < \infty$$

- a. Write an expression for the autocorrelation matrix Φ_{ff} .
 - b. Write a quadratic expression and solve for the eigenvalues of Φ_{ff} .
 - c. Prove that the ratio of the minimum to the average eigenvalue is
- $$\lambda_{\min}/\lambda_{\text{av}} = 1 - \cos(2\pi\nu)$$
- d. Show (in terms of time constants) why the above expression implies increasingly nonideal convergence of the LMS algorithm as ν decreases.
14. Verify the result of Exercise 13 by plotting LMS learning curves for frequencies $\nu = 0.1$ and $\nu = 0.2$ Hz-s and $k = 0:99$, using the signals in Exercise 11 in the configuration of Figure 9.15.
 15. This exercise illustrates misadjustment using the LMS algorithm with $u = 0.05$ and the configuration in Figure 9.9 with the following signals:

$$s_k = \sqrt{2} \sin(2\pi[0:2000]/12)'; \quad n_k = \text{white Gaussian noise}; \quad \sigma_n^2 = 1$$

- a. Set the random number seed to 123, and find the optimum weight vector using actual signals. Compare with the theoretical optimum in Figure 9.2, which was $b_{\text{opt}} = [\sqrt{3}, -1]$.

- b. Plot the error contours on the weight plane as shown in Figure 9.10.
 - c. Run 2000 iterations of the LMS algorithm with $u = 0.05$, using b_{opt} for the initial weight vector. Plot a point for the weight vector at each iteration, and complete the plot in Figure 9.10.
 - d. Compute the theoretical misadjustment for this case.
 - e. Compute the misadjustment using the error signal, e_k , in this example, and compare the result with (d).
16. This exercise tests the theoretical misadjustment formula (9.72) using the LMS algorithm in a manner similar to Exercise 15, that is, with the configuration in Figure 9.9 and with the following signals:

$$s_k = \sqrt{2} \sin(2\pi[0:2000]/12)';$$

$$n_k = \text{white uniform noise}; \sigma_n^2 = 0.05, \mu_n = 0$$

(Note: This exercise involves a total of 1.5 million LMS iterations. If your computer is too slow, use a compiled program to obtain the 16 data points, and then plot these using MATLAB.)

The objective is to start the LMS algorithm at b_{opt} and obtain misadjustments experimentally at each of 16 values of u :

$$u = \frac{0.05n}{15}; \quad n = 0, 1, \dots, 15$$

The experimental misadjustment at $u = 0$ is zero, because no adjustment is allowed in this case. At each value of u greater than zero, make 10 runs, each with 10^4 LMS iterations. At the beginning of the n th run, set the weights to $b_{\text{opt}} = [\sqrt{3}, -1]$, and add a phase shift of $2m\pi/10$ to the argument of s_k so the signal begins at a different point in its cycle. After all 10 runs, compute the MSE and the misadjustment (M) based on the theoretical minimum MSE. Finally, plot M vs. u , and obtain a result similar to Figure 9.11.

References

1. Widrow, B. and Hoff, M.E., Jr., Adaptive switching circuits, 1960 IRE WESCON Convention Record, Part 4, 96, 1960.
2. Widrow, B. et al., Adaptive noise canceling: principles and applications, *Proc. IEEE*, 63, 1692, Dec. 1975.
3. Widrow, B. et al., Adaptive antenna systems, *Proc. IEEE*, 55, 2143, Dec. 1967.
4. Haykin, S.S., *Adaptive Filter Theory*, 2nd ed., Prentice Hall, Englewood Cliffs, NJ, 1991.

5. Widrow, B. and Stearns, S.D., *Adaptive Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1985.
6. Treichler, J.R., Johnson, C.R., Jr., and Larimore, M.G., *Theory and Design of Adaptive Filters*, Texas Instruments, Inc. and John Wiley & Sons, New York, 1987.
7. Honig, M.L. and Messerschmitt, D.G., *Adaptive Filters*, Kluwer Academic Publishers, Norwell, MA, 1984.
8. Atal, B.S. and Hanauer, S.L., Speech analysis and synthesis by linear prediction of the speech wave, *J. Acoustical Soc. Amer.*, 50, 637, 1971.
9. Makhoul, J., Spectral analysis of speech by linear prediction, *IEEE Trans. Audio and Electroacoustics*, 21, 140, June 1973.
10. Landau, A.I., *Adaptive Control: The Model Reference Approach*, Dekker, 1979.
11. Yonig, D.H., Ahmed, N., and Carter, G.C., Magnitude-squared coherence function estimation: an adaptive approach, *IEEE Trans. Acoustics, Speech, and Signal Processing*, 31, 137, Feb. 1983.
12. Widrow, B., McCool, J.M., and Medoff, B., Adaptive control by inverse modeling, *12th Asilomar Conference on Circuits, Systems, and Computers*, 90, Nov. 1978.
13. Widrow, B., Shur, D., and Shaffer, S., On adaptive inverse control, *15th Asilomar Conference on Circuits, Systems, and Computers*, 185, Nov. 1981.
14. Gersho, A., Adaptive equalization of highly dispersive channels for data transmission, *Bell System Tech. J.*, 55, Jan. 1969.
15. Lucky, R.W., Techniques for adaptive equalization of digital communication systems, *Bell System Tech. J.*, 255, Feb. 1966.
16. Monzingo, R.A. and Miller, T.W., *Introduction to Adaptive Arrays*, John Wiley & Sons, New York, 1980.
17. Cowan, C.F.N. and Grant, P.M., Eds., *Adaptive Filters*, Prentice Hall, Englewood Cliffs, NJ, 1985.
18. Widrow, B. et al., Stationary and nonstationary learning characteristics of the LMS adaptive filter, *Proc. IEEE*, 64, 1151, Aug. 1976.
19. White, S.A., A nonlinear digital adaptive filter, *14th Asilomar Conference on Circuits, Systems and Computers*, 350, Nov. 1980.
20. David, R.A., *IIR Adaptive Algorithms Based on Gradient Search Techniques*, Ph.D. Thesis, Stanford University, Stanford, CA, Aug. 1981.
21. Feintuch, P.L., An adaptive recursive LMS filter, *Proc. IEEE*, 1622, Nov. 1976.
22. Larimore, M.G., Treichler, J.R., and Johnson, C.R., Jr., SHARF: an algorithm for adapting IIR digital filters, *IEEE Trans. Acoustics, Speech, and Signal Processing*, 28, 428, Aug. 1980.
23. White, S.A., An adaptive recursive digital filter, *9th Asilomar Conference on Circuits, Systems, and Computers*, 21, Nov. 1975.
24. Widrow, B. and McCool, J.M., A comparison of adaptive algorithms based on the methods of steepest descent and random search, *IEEE Trans. Antennas and Propagation*, 24, 615, Sep. 1976.
25. Etter, D.M. and Masakawa, M.M., A comparison of algorithms for adaptive estimation of the time delay between sampled signals, *Proc. ICASSP-81*, 1253, Mar. 1981.
26. Ahmed, N. et al., A short-term sequential regression algorithm, *IEEE Trans. Acoustics, Speech, and Signal Processing*, 27, 453, Oct. 1979.
27. Bershad, N.J. and Feintuch, P.L., Analysis of the frequency domain adaptive filter, *Proc. IEEE*, 67, 1658, Dec. 1979.
28. Clark, G.A., Mitra, S.K., and Parker, S.R., Block implementation of adaptive digital filters, *IEEE Trans. Circuits and Systems*, 28, 584, June 1981.

29. Clark, G.A., Parker, S.R., and Mitra, S.K., A unified approach to time- and frequency-domain realization of FIR adaptive digital filters, *IEEE Trans. Acoustics, Speech, and Signal Processing*, 31, 1073, Oct. 1983.
30. Dentino, M.J., McCool, J., and Widrow, B., Adaptive filtering in the frequency domain, *Proc. IEEE*, 66, 1658, Dec. 1978.
31. Ferrara, E.R., Fast implementation of LMS adaptive filters, *IEEE Trans. Acoustics, Speech, and Signal Processing*, 28, 474, Aug. 1980.
32. Itakura, F. and Saito, S., Digital filtering techniques for speech analysis and synthesis, *Proc. 7th International Congress on Acoustics*, 261, 1971.
33. Makhoul, J., A class of all-zero lattice digital filters: properties and applications, *IEEE Trans. Acoustics, Speech, and Signal Processing*, 26, 304, Aug. 1978.
34. Makhoul, J., Stable and efficient lattice methods for linear prediction, *IEEE Trans. Acoustics, Speech, and Signal Processing*, 25, 423, Oct. 1977.
35. Griffiths, L.J., An adaptive lattice structure for noise-canceling applications, *Proc. ICASSP-78*, 87, Apr. 1978.
36. Clark, G.A. and Rodgers, P.W., Adaptive prediction applied to seismic event detection, *Proc. IEEE*, 69, 1166, Sep. 1981.
37. Griffiths, L.J., Smolka, F.R., and Trembly, L.D., Adaptive deconvolution: a new technique for processing time-varying seismic data, *Geophysics*, 742, June 1977.
38. Makhoul, J. and Viswanathan, R., Adaptive lattice methods for linear prediction, *Proc. ICASSP-78*, 83, May 1978.
39. Makhoul, J.L. and Cosell, L.K., Adaptive lattice analysis of speech, *IEEE Trans. Acoust. Speech Signal Process.*, ASSP-29, 654, June 1981.
40. Widrow, B. and Walach, E., On the statistical efficiency of the LMS algorithm with nonstationary inputs, *IEEE Trans. Inform. Theory*, IT-30, 211, Mar. 1984.
41. Zeidler, J.R. et al., Adaptive enhancement of multiple sinusoids in uncorrelated noise, *IEEE Trans. Acoust. Speech Signal Process.*, ASSP-26, 240, June 1978.
42. Thomas, G.B., Jr., *Calculus and Analytic Geometry*, 4th ed., Addison-Wesley, Reading, MA, 1968, chap. 10.
43. Luenberger, D.G., *Introduction to Linear and Nonlinear Programming*, Addison-Wesley, Reading, MA, 1973, chap. 7.
44. Lim, J.S. and Oppenheim, A.V., Eds., *Advanced Topics in Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1988, chap. 5.
45. Johnson, C.R., Jr., *Lectures on Adaptive Parameter Estimation*, Prentice Hall, Englewood Cliffs, NJ, 1988.

10

Signal Information, Coding, and Compression

10.1 Introduction

In this chapter, we first discuss the measurement of signal *information content* as a basis for the rest of the subjects in the chapter. Then, we will examine topics applicable to the encoding and compression of signals, that is, waveforms and images. Encoding and compression¹⁻⁷ are broad subjects, and our goal here is to introduce some of the basic techniques and show how they apply.

Signal coding and compression seem destined always to be necessary parts of the world of DSP. Every time the technology offers twice the amount of storage capacity, users discover they have more than twice the amount of information to store. When the capacity of a communication link is increased, the demand for channel capacity seems to increase in proportion. Thus, there is always the requirement to render signals in ways that reduce the related storage capacity or communication bandwidth.

The objective here is to *reduce* or *compress* the number of *bits* (binary digits) in the representation of a signal or image, in other words, to remove unnecessary or redundant information from a signal in ways that reduce its size and yet allow its restoration.

Note that just the opposite is done when signals are encoded for *error detection* and *error correction*. In these cases, redundant information is deliberately added to the signal in order to allow its verification. For example, you could append the sum of all the samples in a frame of waveform data before sending the frame, thus allowing the receiver to detect an error in one of the samples. Simple and practical coding methods for error detection and correction such as Hamming and cyclic codes^{3,5} are well known, but these are subjects usually not covered in a course on DSP, and again, our interest here is in *removing* redundant information from a signal rather than adding it.

Another reason for our interest in coding and compression lies in the close relationship between these subjects and the *encryption* of signals and other data. Encryption is a process where data is transformed in ways such that recovery

is (ideally) not possible except for those who know the transformation. Generally, a transformation is easier to discover when the data contains redundant information, such as a repeated sequence of samples. Thus, although not always done, it is generally best to remove redundant information from data before encrypting it.

When we discuss coding and compression, special terms are commonly used. In coding theory, a sample of a waveform or a pixel in an image becomes a *symbol*, and in some kinds of coding, there is a *symbol code* associated with each symbol. Furthermore, the waveform or image must be *discrete*, with a finite range of values. The symbol set for a *continuous* waveform, however small its range of values, would be infinite. Symbol codes are introduced in Section 10.4.

10.2 Measuring Information

The *bit* (binary digit) is the basic unit we use to measure information. The bit has two possible *states* (zero and one). Other familiar measures are the *byte* (eight bits, $2^8 = 256$ states), which is the basic element in many storage and computing systems, and the computer *word*, which may consist of 16, 32, or 64 bits.

The common measure of the amount of *information* in a waveform or image (or in any similar set of discrete data) is essentially the log base 2 ($\log_2$) of its a priori probability of occurrence. Consider the example in Figure 10.1, which illustrates two possible discrete waveforms in a particular time frame. At each

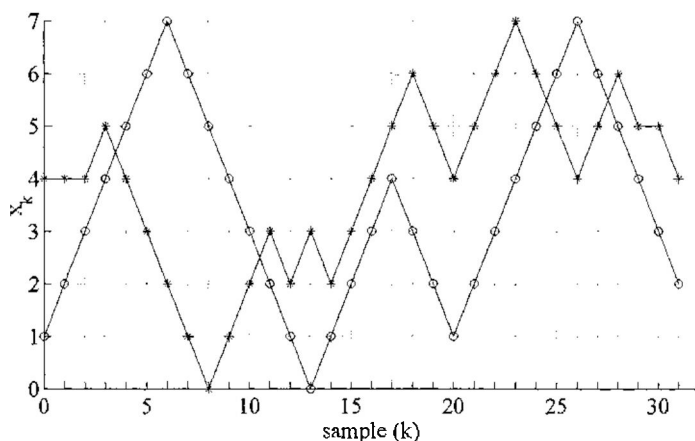


FIGURE 10.1

Two possible waveform vectors out of a large ensemble of different vectors. Each sample in a vector has eight possible values, and there are 32 samples in a vector, so the ensemble consists of 8^{32} vectors.

time step (k) in the frame, a given waveform may have any of eight different amplitudes (states), that is, any sample value from 0 through 7. It follows that any pair of samples may exist in $8^2 = 64$ different states, any vector of three samples in 8^3 states, etc. Therefore, the number of different waveform vectors that could occupy the frame in Figure 10.1 with 32 states is

$$\text{Number of possible vectors} = 8^{32} \quad (10.1)$$

Suppose all 8^{32} waveforms are equally likely. Then, the probability of each possible waveform is 8^{-32} , and as soon as we are given one waveform vector, such as one of the two shown in Figure 10.1, we have gained the amount of information in the vector, which is

$$\text{Information in a vector} = -\log_2(8^{-32}) = 96 \text{ bits} \quad (10.2)$$

The term used for this information measure is *entropy* (H). It is used in the sense that, in this example, the identification of a particular waveform results in the entropy (uncertainty) being reduced by 96 bits. Thus, our basic measure of information, or entropy, which is due to C.E. Shannon,¹ is

Entropy of event with probability P: $H = -\log_2 P$
--

(10.3)

Now suppose, instead of defining an “event” to be an occurrence of a complete vector, we define an event to be the selection of one of the $K = 8$ possible *symbols* (values of x_k) in Figure 10.1, and let P_n represent the probability of this event, as discussed in Chapter 7, Section 7.2. Then, it is reasonable to extend (10.3) and define the average or expected symbol entropy:

Average entropy (bits/symbol): $H = -\sum_{n=0}^{N-1} P_n \log_2 P_n$
--

(10.4)

If all waveform vectors are equally likely, as we have assumed so far, then all eight symbols in Figure 10.1 must also be equally likely with $P_n = 1/8$, and hence, $H = 3$ bits/symbol. Thus, the entropy of a vector with 32 equally likely symbols is $3(32) = 96$ bits, and we see that the entropy measure (10.4) agrees with our idea of accumulating “information.” As each successive symbol becomes known, our uncertainty about the waveform decreases by 3 bits until, after 32 symbols are known, we have a total of 96 bits of information.

There are different ways to apply the entropy measure (10.4). First, the measure is applicable to all kinds of symbol sets (letters, numbers, etc.), and all kinds of information besides just waveforms and images, which are our primary interest here.

Second, when the entropy measure is applied generally to the measurement of information and specifically to the design of codes for signal compression, the determination of the symbol probabilities (P_n) depends on the application. For example, suppose we are measuring the information in the single sentence you are now reading, and P_{101} stands for the probability of the ASCII symbol *e*. We could estimate P_{101} in two ways: first, as the relative occurrence of *e* in this book or in all literature, and second, as the relative occurrence of *e* in the sentence. If we wanted a valid measure of information, we would choose the first way, but if we were interested in compressing just the sentence as an isolated message, we would choose the second, as explained further in the next two sections.

10.3 Two Ways to Compress Signals

In this section, we discuss briefly two different and essentially independent ways to compress a signal or image. For this discussion, we take one of the waveforms in Figure 10.1, say the waveform beginning with $x_0 = 1$, and assume this waveform is to be encoded with fewer than 96 bits and stored in a database. Exact retrieval must be possible knowing only the general rules used for encoding. The waveform is plotted by itself in Figure 10.2.

We first notice that the symbols, that is, waveform values in Figure 10.2, are not equally likely. Using the *freq* function defined in Chapter 7 (7.3), we have the following:

Symbol (n) in x :	0	1	2	3	4	5	6	7
Frequency ($f_r(n)$):	$\frac{1}{32}$	$\frac{4}{32}$	$\frac{6}{32}$	$\frac{6}{32}$	$\frac{5}{32}$	$\frac{4}{32}$	$\frac{4}{32}$	$\frac{2}{32}$

(10.5)

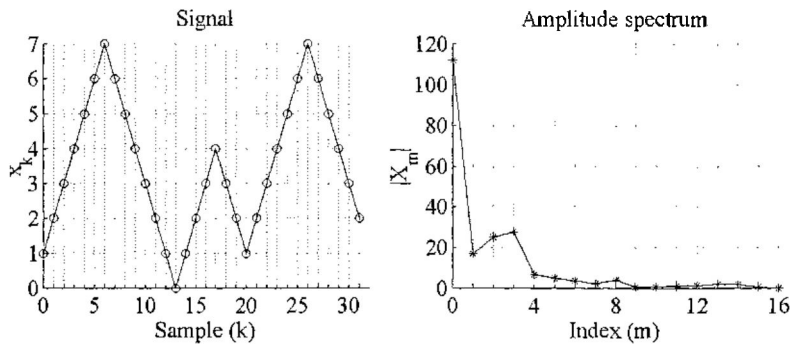


FIGURE 10.2
One of the signal vectors in Figure 10.1 and its amplitude spectrum.